

Autonomous Articulated Vehicle Control via Deep Reinforcement Learning and High-Dimensional Perception

Benjamin Schnapp

June 24, 2026

Contents

1	Introduction	7
1.1	Motivation	7
1.2	Problem Definition	8
1.3	Thesis Contributions	8
1.4	Institutional Context	9
1.5	Thesis Organisation	9
2	Literature Review	10
2.1	Articulated Vehicle Control	10
2.1.1	Classical Stabilisation Techniques	10
2.1.2	Kinematic Models	11
2.2	Reinforcement Learning in Autonomous Driving	11
2.2.1	Policy Gradient Methods	11
2.2.2	Transformer-Based Methods	11
2.3	Perception and Representation Learning	12
2.3.1	Convolutional Neural Networks for Driving	12
2.3.2	Autoencoders for State Compression	12
2.4	Safety in Autonomous Systems	13
2.5	Benchmarking Against Recent Waterloo Theses	13
3	Modeling, Simulation, and Problem Formulation	15
3.1	Tractor Dynamics	15
3.2	Trailer Articulation and Reverse Instability	17
3.3	Challenges of Reverse Maneuvering	18
3.4	Model-Based Control Formulation	19
3.5	Simulation Environment	19
3.6	Environment Tasks	20
3.6.1	Line Following	20
3.6.2	Obstacle Avoidance	21
3.7	Vehicle Parameters	21
3.8	Action Space	22

3.9	Problem Formulation	22
3.10	Testing Protocols	23
3.10.1	Sim-to-Real and ROS2 Transition Strategy	23
3.10.2	Lane Following Performance	24
3.10.3	Obstacle Avoidance	25
4	Learning Framework, Observation Design, and Reward Formulation	26
4.1	Observation Modalities	26
4.2	Bird’s-Eye View Observation	26
4.3	CNN Encoder	27
4.4	Autoencoder Latent Space	28
4.5	UNet Encoder Variant	29
4.6	Lidar Observation	29
4.7	Observation Space Summary	29
4.8	Twin Delayed Deep Deterministic Policy Gradient (TD3)	30
4.8.1	Reward Function	30
4.8.2	Multiplicative vs. Additive Reward Structures	32
4.9	Hyperparameter Optimisation	33
4.9.1	Representation Pretraining	33
4.9.2	Classical Controller Tuning	33
4.10	Failure Replay Curriculum	34
5	Results and Discussion	35
5.1	Path Following Performance	35
5.2	Observation Space Ablation	37
5.3	Lidar Beam-Count Ablation	38
5.4	Reward Formulation Ablation	39
5.5	Statistical Rigor and Generalisation	40
5.6	Training Strategy: Failure Replay Curriculum	41
5.7	Reverse Maneuvering Success	42
5.8	Perception Robustness	42
5.9	Obstacle Navigation	43
5.10	Benchmark Comparison	43
5.11	Failure Analysis and Deployment Relevance	44
6	Sim-to-Real Deployment	46
6.1	Overview	46
6.2	Hardware Platform	46
6.2.1	Tractor Vehicle	46
6.2.2	Trailer	47

6.2.3	Sensor Suite	47
6.3	Software Architecture	48
6.3.1	Middleware and Framework	48
6.3.2	Relationship to the Conventional Articulated Pipeline	48
6.3.3	RL Bridge Node Architecture	48
6.3.4	Hitch Angle Estimation	49
6.3.5	Trailer State Estimation	50
6.3.6	Actuator-Delay Compensation: Smith Predictor	51
6.4	Sim-to-Real Gap Analysis	52
6.4.1	Actuation Dynamics	52
6.4.2	Hitch-Angle Sensing	52
6.4.3	Path Distribution	53
6.4.4	Jackknife Recovery	53
6.4.5	Localisation Dependency	53
6.5	High-Fidelity Geographic Simulation	54
6.6	Infrastructure Sensor Nodes	54
6.7	Deployment Results	55
6.8	Summary	55
7	Conclusion	57
7.1	Summary of Findings	57
7.2	Limitations	58
7.3	Future Work	59
A	Simulation Software Architecture	61
B	Full Hyperparameter Tables	62
C	Additional Plots	63
D	Reward Function Derivation	64

List of Figures

5.1	Learning curves (mean episode return $\pm$ 1 s.d.) for forward lane-following: observation space ablation. Shaded bands show one standard deviation across evaluation episodes at each checkpoint.	37
5.2	Learning curves for reverse lane-following: observation space ablation. . .	37
5.3	Learning curves for forward lane-following: reward formulation ablation.	39
5.4	Learning curves for reverse lane-following: reward formulation ablation. .	39
5.5	Learning curves for reverse lane-following: failure replay curriculum vs random reset baseline. Both agents use identical hyperparameters, observation space, and reward formulation.	41
6.1	Planned infrastructure sensor-node concept: fixed off-vehicle LiDAR units give a bird's-eye view of obstacles around the articulated vehicle, covering the region behind the trailer that the onboard sensors cannot observe. . .	55

List of Tables

1.1	MASc thesis requirements summary	9
2.1	Selected recent MASc theses, University of Waterloo Mechatronics Engineering	13
3.1	Vehicle parameters used in simulation	21
3.2	Lane following experiment definitions	24
4.1	Observation space components	29
5.1	Planned run matrix for the experimental campaign	36
5.2	Forward lane-following: observation space ablation (dense reward, no obstacles, 30 eval episodes).	38
5.3	Reverse lane-following: observation space ablation (dense reward, no obstacles, 30 eval episodes).	38
5.4	Lidar beam-count ablation on forward lane-following (dense reward, no obstacles, 30 eval episodes).	39
5.5	Forward lane-following: reward formulation ablation (lidar-24 obs, no obstacles, 30 eval episodes).	40
5.6	Reverse lane-following: reward formulation ablation (lidar-24 obs, no obstacles, 30 eval episodes).	40
5.7	Failure replay curriculum ablation on reverse lane-following (lidar-24 obs, dense reward, 30 held-out eval episodes).	42
5.8	Forward obstacle-avoidance: TD3 performance by observation space (dense reward, 5–10 obstacles per episode, 30 eval episodes).	43
5.9	Reverse obstacle-avoidance: TD3 performance by observation space (dense reward, 5–10 obstacles per episode, 30 eval episodes).	43
5.10	Forward lane-following: controller benchmark (no obstacles, 50 eval episodes, held-out seeds).	43
5.11	Reverse lane-following: controller benchmark (no obstacles, 50 eval episodes, held-out seeds).	44
5.12	Recommended experiment matrix for the thesis	45

B.1	TD3 Hyperparameters	62
-----	-------------------------------	----

Chapter 1

Introduction

1.1 Motivation

The increasing demand for autonomous logistics and the growth of large-scale goods transportation have positioned articulated tractor-trailer vehicles as a critical target for autonomous control research. The transition from rigid-body autonomous navigation to the control of articulated tractor-trailer systems represents a significant escalation in mechanical complexity, perception requirements, and control instability, particularly during reverse maneuvering.

Unlike single-body vehicles, tractor-trailer systems introduce non-holonomic constraints at the hitch point that fundamentally alter the stability characteristics of the vehicle. In reverse motion, the articulation angle γ between the tractor and trailer exhibits open-loop instability: absent precise closed-loop control, any perturbation grows unbounded, culminating in the well-known jackknife condition from which no steering correction can recover the trailer’s alignment. Classical approaches such as Nonlinear Model Predictive Control (NMPC) can manage this instability by optimising over a prediction horizon, but their computational demands and reliance on accurate system models limit their applicability in unstructured or sensor-rich environments.

This motivates learning-based controllers that map sensor observations directly to actuation at fixed computational cost. Their practical value, however, ultimately rests on a question that a purely simulated study cannot answer on its own: whether a policy trained entirely in simulation can be transferred to, and run on, a physical articulated vehicle. This thesis pursues both halves of that question—the design and evaluation of the learned controller in simulation, and its deployment on real hardware.

1.2 Problem Definition

The core challenge addressed in this thesis is twofold. First, the kinematic and dynamic complexity of tractor-trailer systems makes it difficult to design reward functions and control policies that simultaneously optimise path tracking performance and hitch stability. Second, real-world autonomous deployment requires perception of lane boundaries and obstacles directly from high-dimensional sensor data, rather than from pre-computed symbolic state representations.

Existing learning-based path-following studies on rigid-body vehicles have shown that continuous-control reinforcement learning can achieve performance competitive with classical baselines under nominal conditions, but robustness degrades when the vehicle model, operating conditions, or path geometry become more demanding. Those limitations are amplified for articulated systems, where reverse instability and hitch-angle growth impose a much narrower margin for control error.

1.3 Thesis Contributions

This thesis addresses these challenges through the following contributions:

- developing a kinematic and dynamic model of a tractor-trailer system, including hitch instability analysis for reverse manoeuvring;
- designing a vision-based RL observation space using CNN encoders and autoencoders operating on bird’s-eye-view (BEV) images rendered from a custom Pygame simulation environment;
- engineering a reward function that balances cross-track error, heading error, hitch articulation stability, and collision avoidance;
- implementing and benchmarking Twin Delayed Deep Deterministic Policy Gradient (TD3) agents in lane-following and obstacle-avoidance tasks within a custom Gymnasium-based simulation environment;
- comparing the learned policies against tuned classical control baselines, including pure-pursuit, PID, and MPC, on matched articulated path-tracking tasks;
- deploying the trained forward and reverse TD3 policies directly on a physical articulated platform (an AgileX Hunter SE) through a stripped-down Autoware stack—the “RL bridge”—supported by an onboard LiDAR-based hitch-angle estimator that requires no trailer-mounted sensing and a Smith-predictor compensator for actuator delay (chapter 6).

The research aligns with the objectives of the Mechatronics Vehicle Systems (MVS) Lab and the WATonoBus project at the University of Waterloo.

1.4 Institutional Context

This work is submitted in partial fulfilment of the requirements for a Master of Applied Science (MASc) degree in the Department of Mechanical and Mechatronics Engineering at the University of Waterloo. The degree requires six terms of full-time registration, completion of 3–5 graduate-level courses, appointment of two additional faculty readers, and a mandatory 15-day public display period. The target completion date is early July, with a seminar scheduled for late July.

Table 1.1: MASc thesis requirements summary

Element	Requirement
Research Scope	Substantial problem extending beyond existing laboratory results
Literature Review	Comprehensive assessment of classical control and modern AI approaches
Validation	Simulation-based empirical evidence across multiple environment configurations,
Length/Format	Typically 80–120 pages; GSPA formatting and UWSpace submission
Committee	Supervisor plus two faculty readers; formal display period

1.5 Thesis Organisation

The remainder of this thesis is structured as follows. Chapter 2 reviews classical and learning-based control methods for articulated vehicles, alongside relevant perception and safety literature. Chapter 3 derives the tractor-trailer model, describes the custom Pygame/Gymnasium simulation environment, and formulates the control problem as an MDP. Chapter 4 presents the TD3 learning framework, observation spaces, reward design, representation-learning variants, and training methodology. Chapter 5 presents and discusses the simulation experiments. Chapter 6 carries the learned controller onto a physical tractor-trailer platform, describing the stripped-down Autoware/ROS2 integration that runs the policies, the onboard LiDAR hitch-angle estimator, and the Smith-predictor delay compensation. Chapter 7 summarises the findings and outlines future work.

Chapter 2

Literature Review

This chapter reviews the two methodological traditions on which the thesis draws—classical model-based control of articulated vehicles, and learning-based control with high-dimensional perception—together with the safety considerations that bound both. The intent is not an exhaustive survey but a focused positioning of the learned controller developed in later chapters against the established analytical baselines it is ultimately measured against.

2.1 Articulated Vehicle Control

2.1.1 Classical Stabilisation Techniques

Classical articulated vehicle control methods address the hitch instability through structured model-based approaches:

- **PID control:** Simple to implement and computationally negligible, but incapable of anticipating the jackknife condition. Gains must be re-tuned for different payloads and path curvatures.
- **Model Predictive Control (MPC):** Optimises steering over a receding horizon subject to state and input constraints. Linear MPC can enforce hard bounds on the articulation angle γ .
- **Nonlinear MPC (NMPC):** Captures the full nonlinear kinematics including reverse instability. Optimises steering over a receding horizon subject to state and input constraints, enabling hard bounds on the articulation angle γ , but with higher computational cost than linear methods.
- **Active hitch stabilisation:** Applies an additional actuated degree of freedom at the hitch to damp articulation oscillations. Research at Waterloo into trailer sway control and active hitches demonstrates that lateral articulation at the hitch can

mitigate oscillations, but for a standard autonomous tractor-trailer this must be managed purely through the tractor’s steering angle δ .

Of these, a Linear Quadratic Regulator (LQR) and MPC acting on a *linearised* tractor-trailer state-space model form the conventional, well-understood baseline against which this thesis benchmarks its learned controller; the underlying model is derived in chapter 3, and the same model-based pipeline reappears as the standard alternative in the deployment discussion of chapter 6.

2.1.2 Kinematic Models

The bicycle model provides a baseline for rigid-body longitudinal and lateral dynamics. For an articulated system, the state vector must be expanded to include the trailer heading and articulation angle, as detailed in Chapter 3.

2.2 Reinforcement Learning in Autonomous Driving

Where classical controllers encode the vehicle model explicitly and tune a fixed control law, learning-based methods infer a policy directly from interaction with the environment, trading analytical guarantees for the ability to optimise a control policy end-to-end against a task reward.

2.2.1 Policy Gradient Methods

Deep Deterministic Policy Gradient (DDPG) is an off-policy actor-critic algorithm designed for continuous action spaces. It is one of the canonical deterministic policy-gradient methods used for end-to-end vehicle control, but it is known to suffer from critic overestimation bias and training instability.

Proximal Policy Optimisation (PPO) offers improved stability over DDPG through clipped probability ratios, but its on-policy sample efficiency is lower, which is a significant consideration when simulation rollouts are expensive.

This thesis adopts Twin Delayed DDPG (TD3), a direct successor to DDPG that targets precisely these weaknesses—critic overestimation and update instability—through twin critics, delayed actor updates, and target-policy smoothing; the algorithm and its role here are detailed in chapter 4.

2.2.2 Transformer-Based Methods

Decision Transformers (DT) reformulate reinforcement learning as sequence modelling: an action is predicted from a context window of past states, actions, and returns-to-go

using a causal self-attention mechanism. In vehicle-control settings, DT-style models can represent longer temporal dependencies than shallow actor-critic policies, but their performance depends strongly on the diversity and quality of the offline trajectory dataset. In particular, datasets dominated by nominal path-tracking behaviour can lead to poor recovery performance from high-error states. Effective DT training for articulated vehicles would therefore require explicit coverage of recovery manoeuvres and unstable reverse-driving configurations. The present work retains an actor-critic formulation rather than a sequence model, but this same need for recovery coverage motivates the failure-replay curriculum introduced in chapter 4.

2.3 Perception and Representation Learning

A second axis of the thesis concerns how the agent perceives its surroundings—whether a compact, hand-engineered state vector or a learned representation of a bird’s-eye view image yields better and more transferable control.

2.3.1 Convolutional Neural Networks for Driving

CNNs have become the standard front-end for autonomous driving perception, extracting spatial features from camera images and occupancy grids. End-to-end learning approaches train CNNs directly from sensor inputs to control outputs, allowing the network to learn which visual features are most relevant for the control task. In the simulation setting of this thesis, CNNs process bird’s-eye view images rendered from the Pygame environment to extract lane geometry and obstacle information.

2.3.2 Autoencoders for State Compression

Autoencoders compress high-dimensional observations into a compact latent representation, mitigating the curse of dimensionality for RL training. By pre-training an autoencoder on images collected from simulation rollouts, the encoder learns to capture essential geometric features of the road and obstacles. The RL agent then operates on the latent vector z , which reduces policy network parameters, accelerates convergence, and improves generalisation across environments. The thesis treats such learned encoders as one end of a representation spectrum, with engineered state vectors at the other, and compares the two directly in chapter 4.

2.4 Safety in Autonomous Systems

Safety of the Intended Functionality (SOTIF) defines the safety boundary for autonomous systems operating under nominal sensor and actuator conditions. For articulated vehicles, a key safety concern is the jackknife condition: the RL reward function must penalise large articulation angles to discourage the agent from entering unstable configurations. Control Barrier Functions (CBFs) offer a formal mechanism to overlay hard safety constraints on top of a learned policy, and are identified as a direction for future work beyond the simulation-based experiments of this thesis.

2.5 Benchmarking Against Recent Waterloo Theses

To contextualise the expected level of rigor, Table 2.1 summarises recent MASc theses from the Mechatronics domain at the University of Waterloo.

Table 2.1: Selected recent MASc theses, University of Waterloo Mechatronics Engineering

Author (Year)	Topic	Methodology	Rigor Highlights
Joseph (2025)	Gaze-enabled grasping assistance	ROS2, Kinova Arm, Meta Quest Pro	User study with 30 participants; workload and performance metrics
Thibault (2022)	Humanoid robotic manipulation	EUROBENCH, REEM-C robot	Defined “loco-manipulation”; new manipulability–stability metrics
Hart (2023)	Gait detection via machine learning	Wearable sensors, ternary classification	6 classical + 1 Transformer model; grid-search hyperparameter optimisation
Wei (2021)	Path following for manipulators	Transpose Jacobian control	Free of inverse transformations; validated on 2-DoF and 4-DoF robots
Bickford (2019)	Autonomous bicycle stabilisation	Linear control, Whipple model	Multi-loop architecture; “virtual paths” for high-error recovery

Common themes include strong mathematical modelling foundations and comprehensive validation datasets. For this thesis, a simple “it works” result is insufficient;

the work must quantify stability limits, evaluate performance across varied paths, and compare against classical benchmarks.

Chapter 3

Modeling, Simulation, and Problem Formulation

3.1 Tractor Dynamics

The tractor is represented by a planar single-track (“bicycle”) model, in which the front axle is lumped into one steerable wheel and the rear axle into one fixed wheel. Its global position (x, y) follows from the body-frame longitudinal and lateral velocities (v_x, v_y) and the heading ψ :

$$\dot{x} = v_x \cos \psi - v_y \sin \psi \quad (3.1)$$

$$\dot{y} = v_x \sin \psi + v_y \cos \psi \quad (3.2)$$

Lateral force is generated at each tyre as a function of its slip angle. Under the small-angle assumptions appropriate to the operating envelope, the front and rear slip angles are

$$\alpha_f = \delta - \frac{v_y + l_f \dot{\psi}}{v_x}, \quad \alpha_r = - \frac{v_y - l_r \dot{\psi}}{v_x}, \quad (3.3)$$

where δ is the steering angle. Rather than evaluating the full Pacejka magic-formula tyre curve, the simulator operates in its linear region: for the modest slip angles encountered here the curve is well approximated by a constant cornering stiffness, giving the lateral tyre forces

$$F_{yf} = C_f \alpha_f, \quad F_{yr} = C_r \alpha_r. \quad (3.4)$$

This linearised-Pacejka (constant cornering-stiffness) model is what is implemented in code; it preserves the lateral behaviour relevant to path tracking while avoiding the hard-to-identify parameters of the full nonlinear tyre curve. Collecting the longitudinal, lateral, and yaw-moment balances — with a quadratic aerodynamic drag $F_{\text{drag}} = \frac{1}{2}C_d\rho Av_x^2$ and a net drive force F_x — yields the tractor dynamics

$$m(\dot{v}_x - v_y\dot{\psi}) = F_x - F_{\text{drag}} - F_{yf}\sin\delta, \quad (3.5)$$

$$m(\dot{v}_y + v_x\dot{\psi}) = F_{yf}\cos\delta + F_{yr}, \quad (3.6)$$

$$I_z\ddot{\psi} = l_f F_{yf}\cos\delta - l_r F_{yr}. \quad (3.7)$$

For control design and for the time-stepping used in simulation, these equations are linearised about straight-line motion ($\delta \approx 0$) at a constant reference speed $v_x = v$. With state $\boldsymbol{\xi} = [v_x, v_y, \dot{\psi}]^\top$ and input $\mathbf{u} = [F_x, \delta]^\top$, the linearised dynamics take the time-invariant form $\dot{\boldsymbol{\xi}} = A\boldsymbol{\xi} + B\mathbf{u}$ with

$$A = \begin{bmatrix} -\frac{C_d\rho Av}{2m} & 0 & 0 \\ 0 & -\frac{C_f + C_r}{mv} & \frac{l_r C_r - l_f C_f}{mv} \\ 0 & \frac{l_r C_r - l_f C_f}{I_z v} & -\frac{l_f^2 C_f + l_r^2 C_r}{I_z v} \end{bmatrix}, \quad B = \begin{bmatrix} \frac{1}{m} & 0 \\ 0 & \frac{C_f}{m} \\ 0 & \frac{l_f C_f}{I_z} \end{bmatrix}. \quad (3.8)$$

The continuous system is converted to the discrete update $\boldsymbol{\xi}_{k+1} = A_d\boldsymbol{\xi}_k + B_d\mathbf{u}_k$ by a zero-order-hold discretisation at the simulation step $\Delta t = 0.1\text{ s}$, and it is this discrete map that is advanced at each timestep. The block structure of (3.8) shows that, to first order, the longitudinal channel decouples from the coupled lateral/yaw channel — a property the fixed-speed path-tracking experiments exploit by holding speed constant and studying only the lateral and articulation behaviour.

The tractor is modelled as a planar bicycle model, which captures the dominant lateral and yaw behaviour required for lane-following and low-speed manoeuvring. In this representation the front axle is replaced by a single steerable wheel and the rear axle by a single non-steered wheel. The global position (x, y) evolves from the body-frame longitudinal and lateral velocities (v_x, v_y) and the tractor heading ψ , as shown in Equations 3.1 and 3.2. This state description is sufficient for the control tasks studied in the thesis because the dominant quantities of interest are path-relative position error, heading error, steering angle, and hitch articulation. These kinematics are driven by the tractor's lateral dynamics: tyre slip angles (eq. (3.3)) generate lateral forces through a linearised tyre model (eq. (3.4)), which combine into the longitudinal, lateral, and yaw-

moment balances (eqs. (3.5) to (3.7)) and, after linearisation about a constant reference speed, the discrete state-space model (eq. (3.8)) that the simulator advances at each timestep.

The modelling objective is not to reproduce every high-speed transient of a road vehicle, but to provide a physically meaningful control environment in which steering commands, heading evolution, and articulation dynamics remain coupled. This balance is important for reinforcement learning: an overly simplified kinematic model would make the task unrealistically easy, whereas a full high-fidelity multibody model would obscure the core control-design questions addressed here.

3.2 Trailer Articulation and Reverse Instability

The articulated system adds a trailer body coupled to the tractor through a revolute hitch. The relevant geometry is:

- L_h : signed distance from the tractor rear axle to the hitch point (in both the training model and the hardware platform the hitch sits at the rear axle, so $L_h \approx 0$);
- L_t : distance from the hitch to the trailer axle, i.e. the trailer wheelbase;
- $\gamma = \psi - \psi_t$: the articulation angle between the tractor heading ψ and the trailer heading ψ_t (the same convention used by the deployment in chapter 6).

The hitch point is carried rigidly by the tractor,

$$x_h = x - L_h \cos \psi, \quad y_h = y - L_h \sin \psi, \quad (3.9)$$

and translates with the tractor's longitudinal speed v . Imposing a pure-rolling (no lateral slip) constraint on the trailer axle fixes the trailer yaw rate from the lateral component of the hitch velocity:

$$\dot{\psi}_t = \frac{v}{L_t} \sin(\psi - \psi_t) = \frac{v}{L_t} \sin \gamma. \quad (3.10)$$

The articulation angle therefore evolves as

$$\dot{\gamma} = \dot{\psi} - \frac{v}{L_t} \sin \gamma, \quad (3.11)$$

where the tractor yaw rate $\dot{\psi}$ is supplied by the tractor model of eq. (3.8) (equivalently $\dot{\psi} = (v/L) \tan \delta$ for a purely kinematic tractor of wheelbase $L = l_f + l_r$). The trailer axle pose then follows from the hitch point and the trailer heading,

$$x_t = x_h - L_t \cos \psi_t, \quad y_t = y_h - L_t \sin \psi_t. \quad (3.12)$$

The sign of v in eq. (3.11) is what makes reverse motion qualitatively harder. Linearising about the aligned configuration ($\gamma \approx 0$, so $\sin \gamma \approx \gamma$) and holding the steering-driven tractor yaw fixed, the unforced articulation mode obeys

$$\dot{\gamma} \approx -\frac{v}{L_t} \gamma, \quad \lambda = -\frac{v}{L_t}. \quad (3.13)$$

In forward motion ($v > 0$) the eigenvalue λ is negative: articulation errors decay and the trailer self-aligns behind the tractor. In reverse ($v < 0$) the eigenvalue becomes positive and the same errors grow exponentially toward the jackknife condition. This is an open-loop instability rather than a mere tuning difficulty, and it is the defining feature of reverse articulated manoeuvring — the reason the reverse task demands active hitch stabilisation, trailer-referenced error metrics, and the dedicated reward and curriculum treatments developed in chapter 4.

The trailer introduces an additional geometric state, the articulation angle γ , which measures the relative orientation between the tractor and trailer bodies. This variable is central to articulated-vehicle control because it determines whether the trailer remains aligned with the tractor or diverges toward a jackknife configuration. Equation 3.11 captures how steering angle and longitudinal motion interact through the hitch geometry to change γ over time.

In forward motion, the hitch dynamics are naturally self-correcting over a broad operating range: modest articulation tends to decay as the tractor pulls the trailer back into alignment. In reverse motion, the sign of the longitudinal velocity changes this behaviour qualitatively. Small articulation errors can grow rather than decay, so the driver or controller must actively stabilise the hitch while simultaneously tracking the path. This instability is the defining challenge of reverse articulated manoeuvring and motivates the use of explicit hitch-angle penalties, trailer-referenced tracking metrics, and dedicated reverse-controller formulations throughout the thesis.

3.3 Challenges of Reverse Maneuvering

Reverse motion is characterised by a sign reversal in the kinematic equations, which transforms the stabilising effect of forward velocity into a destabilising one. Successful reverse manoeuvring therefore requires anticipatory control: the steering input must correct developing trailer error before the hitch angle grows large enough to become unrecoverable. In practical terms, the controller must regulate three coupled objectives at once: trailer path tracking, tractor placement, and articulation stability.

This coupling is what distinguishes articulated reverse control from conventional rigid-vehicle lane following. A controller that focuses only on tractor cross-track error can produce apparently reasonable short-term motion while simultaneously driving the hitch

toward instability. For that reason, the reinforcement-learning rewards and classical benchmark metrics used in this thesis prioritise trailer tracking and hitch-angle containment rather than treating them as secondary diagnostics.

3.4 Model-Based Control Formulation

The linearised state-space model of eq. (3.8) is not only a simulation device; it is also the foundation of the classical controllers against which the learned policy is benchmarked, and of the model-based pipeline used in the hardware deployment (chapter 6). Because the pair (A, B) is controllable across the operating-speed range, a linear state-feedback law $\mathbf{u} = -K\boldsymbol{\xi}$ can regulate the tracking states to zero. Choosing the gain K as the linear-quadratic regulator (LQR) that minimises

$$J = \int_0^\infty (\boldsymbol{\xi}^\top Q \boldsymbol{\xi} + \mathbf{u}^\top R \mathbf{u}) dt \quad (3.14)$$

trades tracking accuracy (through the weight Q) against control effort (through R). The block structure of eq. (3.8) leaves the longitudinal and lateral/yaw channels nearly decoupled, so in practice K is given a block-diagonal form that separates speed regulation from steering. This linear design is exact only near its operating point: the pure-pursuit, PID, and MPC baselines tuned in chapter 4 extend the same model-based philosophy to the full nonlinear path-tracking problem, and the deployment’s conventional pipeline (chapter 6) uses an analogous LQR/MPC tracker. The reinforcement-learning controller developed in this thesis is, in effect, a learned alternative to this model-based chain that requires neither an explicit linearisation nor hand-tuned gains.

3.5 Simulation Environment

The training and evaluation environment is a custom 2D simulation built on the Farama Foundation Gymnasium library. Gymnasium provides a standardised API for implementing environments that integrate directly with reinforcement learning algorithm libraries, including Stable Baselines3.

The simulation is implemented using Pygame for rendering and physics integration. At each timestep, the discrete-time tractor-trailer state-space equations are re-evaluated using the previous state and control inputs, updating the vehicle position, heading, hitch angle, and associated error signals. Pygame renders the scene as a 2D top-down view, from which the BEV image observation is captured.

The simulation environment includes:

1. **Vehicle Model:** A combined dynamic tractor (bicycle model with a Pacejka tire

model linearised to constant cornering stiffness, eq. (3.4)) and kinematic trailer with hitch constraints, using Tesla Model S parameters.

2. **Lane Generation:** Randomly generated lane paths using cubic spline interpolation, producing varying curvature paths for training and evaluation.
3. **Obstacle Placement and Local Replanning:** Randomly placed static obstacles within the lane and a local path planner that perturbs the centreline around those obstacles for avoidance experiments.
4. **Collision Detection:** Pygame sprite mask-based collision detection, providing pixel-precise contact sensing between the vehicle body and obstacles or lane boundaries.
5. **BEV Camera:** A configurable bird’s-eye view observation derived from the Pygame render, cropped to a local vehicle-centred region and resized to a 32×32 grayscale image.

3.6 Environment Tasks

The environment implementation is structured around reusable task and observation components. A base lane-following task is combined with different observation heads (state only, lidar, or BEV), while obstacle-aware variants reuse the same task logic through a mixin that adds obstacle generation, local path planning, and obstacle-aware reward evaluation. This avoids maintaining separate forward, reverse, lidar, and BEV implementations with duplicated task logic.

3.6.1 Line Following

The lane-following environment extends the base tractor-trailer environment to present a lane-tracking task. A reference lane is generated at the start of each episode using cubic spline interpolation through randomly placed waypoints. The reward function encourages the agent to keep both the tractor and trailer within the lane boundaries while maintaining progress along the path.

For forward driving, the observation at each timestep may include:

- **vector:** steering angle δ , hitch articulation angle γ , tractor and trailer cross-track and heading errors, and local curvature cues;
- **lidar:** simulated obstacle distances appended to the vector state when lidar sensing is enabled;

- **image**: a 32×32 grayscale BEV crop derived from the tractor rear axle anchor point.

Reverse line-following environments use the same path generation and rendering pipeline, but reverse the longitudinal command and compute reward and tracking terms with the trailer as the leading unit. Reverse BEV and reverse lidar observation classes are implemented explicitly so that perception experiments can be repeated in backward manoeuvres without changing the task definition.

3.6.2 Obstacle Avoidance

The obstacle avoidance environment adds static obstacles within the lane. The agent must navigate the tractor-trailer through the lane while avoiding obstacles, which requires both path-following behaviour and collision avoidance. Instead of following the global lane centreline directly, the environment constructs a local path around the placed obstacles and evaluates tracking error relative to this locally planned path. Episode termination occurs on collision, jackknife ($|\gamma| > 90^\circ$), or successful path completion. In addition to terminal collision penalties, the lane-following base environment applies a per-step proximity penalty near blocked occupancy-grid cells, so both lane boundaries and inserted obstacles discourage the agent before contact occurs.

Forward and reverse obstacle environments are both available. In addition to the lidar-based formulation, the environment includes BEV obstacle-avoidance variants for both travel directions, enabling direct comparison between range-based and image-based obstacle perception within the same local-planning task.

3.7 Vehicle Parameters

The simulation uses Tesla Model S specifications for the tractor parameters:

Table 3.1: Vehicle parameters used in simulation

Parameter	Value	Parameter	Value
m	1500 kg	l_f	1.2 m
I_z	3000 kg m ²	l_r	1.6 m
C_f	80000 N/rad	C_d	0.208
C_r	80000 N/rad	A	2.4 m ²
Δt	0.1 s	ρ	1.225 kg/m ³

3.8 Action Space

The agent controls the vehicle through a continuous two-dimensional action space:

$$\mathbf{a} = [\dot{\delta}, v] \in [-25^\circ/\text{s}, 25^\circ/\text{s}] \times [-5, 15] \text{ m/s} \quad (3.15)$$

where $\dot{\delta}$ is the steering rate (rate of change of steering angle) and v is the commanded speed. This action parameterisation reflects the fact that the steering angle cannot be set instantaneously on a physical vehicle; controlling the rate of change is more physically realistic.

For reverse tasks, the environment constrains the longitudinal command to negative speed while preserving the same steering-rate interface. This keeps the control dimensionality fixed across forward and reverse tasks and allows the same TD3 implementation to be reused.

For evaluation, this full action space is not used identically in every study. In the no-obstacle line-following experiments, the longitudinal speed is held at a fixed operating value (5 m/s forward, -5 m/s in reverse) so that all methods are compared on the same path-tracking problem, leaving the agent to command only the steering rate. In the obstacle-avoidance experiments, speed modulation remains part of the task because the agent must trade progress against collision risk and path geometry.

3.9 Problem Formulation

The control problem is cast as a Markov Decision Process (MDP) $(\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma_d)$ where:

- $\mathcal{S}$: observation space consisting of a vector of state variables (steering angle, articulation angle, cross-track error, heading error) and, for vision-based policies, a 32×32 grayscale BEV image;
- $\mathcal{A}$: continuous action space $[\dot{\delta}, v] \in [-25^\circ/\text{s}, 25^\circ/\text{s}] \times [-5, 15] \text{ m/s}$ (with speed held fixed in the no-obstacle tasks);
- $\mathcal{T}$: transition dynamics simulated in the custom Pygame/Gymnasium environment;
- $\mathcal{R}$: reward function defined in Chapter 4;
- γ_d : discount factor.

Episodes terminate on collision, jackknife ($|\gamma| > 90^\circ$), or successful path completion.

3.10 Testing Protocols

The experimental workflow is scriptable end-to-end. Scenario generation, observation-modality training, trained-policy evaluation, controller tuning, and multi-controller benchmarking are each exposed through standalone scripts. The benchmarking pipeline runs all controllers on the same pre-generated scenario set and logs per-step metrics including tractor and trailer cross-track error, hitch angle, completion, collision, jackknife events, obstacle clearance, and inference latency.

The experiments are treated as two related but distinct problem classes:

1. **Path-tracking control:** forward and reverse lane following without obstacles. In this setting all methods are solving the same task at a common fixed speed, so TD3 can be compared directly against pure-pursuit, PID, and MPC as lateral/path-tracking controllers.
2. **Integrated navigation:** obstacle-avoidance tasks in which the policy must implicitly combine local planning and control. In this setting a direct comparison against controller-only baselines is not fair unless those baselines are also given a local planner.

This distinction is important for experimental validity. A controller that only tracks a supplied path should not be evaluated against an RL policy that must both choose a collision-free local path and execute it, unless the classical baseline is paired with an equivalent planner.

3.10.1 Sim-to-Real and ROS2 Transition Strategy

The learning experiments in this thesis are conducted in simulation, but the software architecture was designed from the outset to port onto a ROS2-based robotics stack — a port that has since been realised on physical hardware (chapter 6). The key design decision is that the policy and controller interfaces are expressed in terms of generic observations and low-level control commands rather than simulator-specific rendering calls. Each control method consumes either a vector observation, a lidar-style range vector, or a BEV image paired with state variables, and produces steering-rate and speed commands. This abstraction is directly compatible with a ROS2 node graph in which sensor drivers publish observations, a policy node performs inference, and a downstream interface node converts the commands into actuator messages.

For no-obstacle path-tracking experiments, the fixed-speed formulation used in the thesis also improves deployment clarity: a real vehicle implementation can treat longitudinal speed regulation as a separate closed-loop subsystem while the learned or analytical controller handles lateral and articulation control. For obstacle-navigation experiments,

the policy retains speed authority because longitudinal modulation is part of the task definition. This separation reduces ambiguity when mapping the thesis experiments onto a real robotic architecture.

The ROS2 transition therefore reduces to replacing only the environment-facing components:

- the simulated state vector with subscribed vehicle-state estimates;
- the simulated lidar or BEV renderer with ROS2 sensor topics or perception outputs;
- the Pygame environment step call with a command publisher targeting steering and speed interfaces;
- the episode reset logic with scenario initialisation and safety supervision on the robot.

The policy network, observation formatting, benchmark metrics, and controller-selection logic can remain largely unchanged. This is important because it turns ROS2 deployment from a full reimplementation problem into an interface-substitution problem.

However, portability should not be overstated. To close the remaining gap, the simulator must be augmented with domain randomisation and system identification so that the learned policy is exposed to uncertainty in tire parameters, friction, mass distribution, actuator delay, sensor noise, and hitch compliance. Several of these factors are taken up directly in the deployment of chapter 6 — notably actuator delay, which is addressed there by a Smith-predictor compensator — but in the absence of systematic randomisation, strong simulation performance alone is not evidence of hardware readiness.

3.10.2 Lane Following Performance

Table 3.2: Lane following experiment definitions

Experiment	Objective	Success Metric
Path Fidelity (Tractor)	Maintain low CTE for tractor on varying curvature paths	Mean Absolute CTE
Path Fidelity (Trailer)	Maintain low CTE for trailer, ensuring full vehicle stays in lane	Mean Absolute CTE
Hitch Stability	Keep articulation angle within safe bounds	Max $ \gamma $ per episode

3.10.3 Obstacle Avoidance

The obstacle avoidance task tests the agent’s ability to navigate through a lane containing static obstacles. Success is measured by collision rate and the agent’s ability to complete the lane while maintaining hitch stability.

Obstacle experiments are framed as a secondary study rather than merged into the primary controller benchmark. Two defensible comparison strategies are available:

- **Planner-controlled comparison:** provide the classical baselines with the same obstacle-aware local path and evaluate only the downstream tracking problem; or
- **End-to-end navigation comparison:** compare the RL obstacle policy against complete planner+controller stacks, not against controllers in isolation.

In the present study, obstacle avoidance is treated as an RL feasibility extension and the main quantitative comparison is reserved for the no-obstacle forward and reverse tracking tasks. Under this structure, no-obstacle experiments use fixed speed for both learned and classical methods, whereas obstacle experiments allow the RL agent to command speed as part of the end-to-end navigation problem.

Chapter 4

Learning Framework, Observation Design, and Reward Formulation

4.1 Observation Modalities

The simulator supports three observation families so that perception can be studied as an ablation rather than a fixed design choice:

- **State-only:** an 8-dimensional vector containing steering angle, hitch angle, tractor cross-track and heading error, trailer cross-track and heading error, and two local curvature terms.
- **State + lidar:** the same 8-dimensional state vector augmented with N normalised range readings, where the beam count $N \in \{8, 16, 24, 32\}$ is varied in the lidar ablation of Chapter 5.
- **State + BEV image:** a dictionary observation with the state vector and a 32×32 grayscale bird’s-eye-view image.

This design allows the thesis to compare hand-engineered geometric sensing against learned visual representations within the same tractor-trailer task. The three families form a deliberate progression—from a fully engineered state vector, through a mid-level geometric range sensor, to raw bird’s-eye-view imagery—so that the marginal value of richer perception can be measured against its added representation-learning and inference cost rather than assumed.

4.2 Bird’s-Eye View Observation

The BEV image is generated by first rendering the normal colour Pygame scene and then applying an observation-only transformation before the image is passed to the policy. The

base renderer is therefore unchanged for human visualisation, while the policy receives a compact grayscale crop derived from the rendered frame. The current BEV observation is a 32×32 single-channel image. It is obtained from a local 20 m square crop of the transformed BEV frame: the forward observation uses the top portion of the local frame to bias the crop toward upcoming path geometry, while the reverse observation uses the bottom portion to bias the crop toward the region behind the vehicle. The image encodes lane markings, obstacle positions, local path geometry, and vehicle geometry, providing spatial context that the vector observation alone cannot capture.

Forward and reverse variants of the BEV environment share the same image size and channel format. They differ only in the crop direction used before resizing and in the interpretation of the vector component. In reverse manoeuvres, the vector component is interpreted with trailer-led tracking errors because the trailer becomes the leading unit during backward motion.

4.3 CNN Encoder

A Convolutional Neural Network (CNN) serves as the front-end of the RL agent, processing the BEV image to extract spatial features relevant to lane tracking and obstacle avoidance. The CNN enables the agent to perceive the environment directly from rendered images, without requiring a pre-defined symbolic representation of every object.

The CNN architecture is implemented as a custom feature extractor within the Stable Baselines3 framework. The current end-to-end image branch is deliberately small, matching the scale of the 32×32 observation: a two-layer convolutional encoder with $1 \rightarrow 5 \rightarrow 5$ channels maps the image to an 80-dimensional feature vector, followed by a linear projection to 64 image features. A separate two-layer multilayer perceptron embeds the 8-dimensional vector state to 64 dimensions. The two branches are concatenated before being passed to the TD3 actor and critic networks, producing a 128-dimensional feature vector for the policy and critic heads. Actor and critic feature extractors are not shared, following the TD3 default, so that critic representation updates do not directly overwrite the actor’s feature representation.

The CNN can be trained end-to-end with the TD3 agent, allowing the feature extractor to learn which visual features are most relevant for path maintenance and collision avoidance. Initial experiments showed that inserting a randomly initialised CNN branch into an otherwise successful state-based policy can harm early TD3 learning: the agent converged to a short-horizon wall-collision behaviour even though the same vector state was sufficient for rapid learning. To isolate this effect, two diagnostic BEV encoders are included. The `state_only` BEV encoder keeps the dictionary observation and multi-input policy path but ignores the image branch, confirming that the BEV environment and multi-input wrapper can still learn from the vector state. The `scaled_cnn` encoder

introduces the image branch gradually using a scalar schedule:

$$\phi(s) = [\lambda_I(k) \phi_I(I), \phi_x(\tilde{x})], \quad (4.1)$$

where ϕ_I is the image encoder, ϕ_x is the vector-state encoder, and $\lambda_I(k)$ ramps from zero to one over training. A second schedule can reduce the non-steering vector components after the image branch has reached full weight:

$$\tilde{x} = [\delta, \lambda_x(k)\gamma, \lambda_x(k)e_y, \lambda_x(k)e_\psi, \lambda_x(k)e_{y,t}, \lambda_x(k)e_{\psi,t}, \lambda_x(k)\kappa_1, \lambda_x(k)\kappa_2]. \quad (4.2)$$

The steering angle δ is intentionally preserved because it is not reliably inferable from the 32×32 BEV image.

4.4 Autoencoder Latent Space

The diagnostics above motivate decoupling representation learning from control. If a randomly initialised image branch can derail early TD3 learning even when the underlying vector state is easily learnable, then supplying the agent with a usable visual representation from the first training step should remove that failure mode. To improve RL training efficiency, an autoencoder can be used to pretrain an image representation for high-dimensional BEV observations before policy optimisation. The encoder $E(\cdot)$ maps the image observation s to a feature vector z , and the decoder $D(\cdot)$ reconstructs the original image:

$$z = E(s) \quad (4.3)$$

$$\hat{s} = D(z) \quad (4.4)$$

The autoencoder is pre-trained on images collected from simulation rollouts, allowing the encoder to learn compact representations of the road geometry and obstacle configurations. BEV images are collected from a mixture of simple pure-pursuit rollouts, already-trained compatible policies, and random actions to increase state coverage during pretraining. After reconstruction training, the encoder weights are transferred into the RL feature extractor and can be frozen or fine-tuned while the policy is trained. This produces an ablation between end-to-end image learning, fixed latent representations learned before RL, and pretrained representations that continue adapting during control learning.

4.5 UNet Encoder Variant

In addition to the CNN autoencoder, a UNet-style autoencoder is used for BEV representation learning. The encoder uses repeated double-convolution blocks with max pooling, followed by global average pooling to produce a compact 128-dimensional image descriptor. During pretraining, the full UNet decoder retains skip connections to improve image reconstruction quality; for RL, only the encoder path is kept and concatenated with the 64-dimensional vector embedding.

This second pretrained encoder provides a stronger structural prior than the plain CNN autoencoder. It is intended to test whether preserving multi-scale spatial detail during representation learning improves downstream control performance, particularly in scenes where obstacle shape and lane boundaries must both be interpreted from the BEV image.

4.6 Lidar Observation

The obstacle environments expose a simulated planar lidar observation by appending normalised obstacle distances to the state vector. In forward driving the lidar pose is attached to the tractor, while in reverse driving it is attached to the trailer and rotated by π so that the sensing direction follows the leading unit. This design keeps the sensing model consistent with the physical task: the front of the articulated system changes when the direction of travel reverses.

4.7 Observation Space Summary

The full observation interface used across experiments is summarised in Table 4.1.

Table 4.1: Observation space components

Component	Description	Role
vector	8-dimensional state: $[\delta, \gamma, e_y, e_\psi, e_{y,t}, e_{\psi,t}, \kappa_1, \kappa_2]$	Provides precise articulation and path-tracking state information
lidar	N normalised obstacle ranges, with $N \in \{8, 16, 24, 32\}$	Encodes local free space around the leading unit without image processing
image	32×32 grayscale BEV crop derived from the Pygame render	Encodes lane layout, local path shape, obstacle positions, and vehicle geometry

Collision detection is performed using Pygame sprite mask overlap, enabling pixel-precise detection of contact between the vehicle body and obstacles or lane boundaries. Episode termination is triggered on collision, jackknife ($|\gamma| > 90^\circ$), or path completion.

4.8 Twin Delayed Deep Deterministic Policy Gradient (TD3)

Twin Delayed Deep Deterministic Policy Gradient (TD3) is an off-policy actor-critic algorithm designed for continuous action spaces. TD3 extends deterministic policy-gradient methods with three key improvements: twin critic networks (to reduce overestimation bias), delayed policy updates (to improve stability), and target policy smoothing (to reduce variance in critic updates). These properties make TD3 a natural fit for articulated-vehicle control: the action space (steering rate and, where enabled, speed) is continuous, a deterministic policy avoids the action-distribution spread that can aggravate the non-minimum-phase reverse dynamics, and the off-policy replay buffer amortises the comparatively high cost of image-based rollouts by reusing every transition many times.

The TD3 actor network $\mu(s \mid \theta^\mu)$ maps states to deterministic actions, while the critic networks $Q_1, Q_2(s, a \mid \theta^{Q_1}, \theta^{Q_2})$ evaluate action quality. The target value is computed using the minimum of the two critic estimates:

$$y = r_t + \gamma_d \min_{i=1,2} Q_i(s_{t+1}, \mu(s_{t+1})) + \epsilon \quad (4.5)$$

where $\epsilon \sim \text{clip}(\mathcal{N}(0, \sigma), -c, c)$ is clipped Gaussian noise added to the target action for smoothing. Parameters are updated by minimising the temporal difference error in the critics and maximising the expected return through the actor using the deterministic policy gradient theorem.

TD3 is implemented using the Stable Baselines3 library, which provides a reliable implementation integrated with the Gymnasium environment API.

4.8.1 Reward Function

The thesis evaluates several reward variants rather than a single scalar shaping rule. Let e_y and e_ψ denote the tractor cross-track and heading errors, $e_{y,t}$ and $e_{\psi,t}$ the trailer cross-track and heading errors, γ the hitch articulation angle, $\dot{x}$ the longitudinal speed, and δ the commanded steering action. The guide controller action is denoted δ_{pp} , with steering normalization $\Delta_\delta = (\delta - \delta_{\text{pp}})/(2\delta_{\text{max}})$. The lane-following base environment also defines a proximity penalty P_{prox} that is subtracted from non-terminal step rewards. This penalty is zero when the vehicle is farther than $d_{\text{prox}} = 2$ m from blocked occupancy-grid cells and

increases quadratically to $P_{\max} = 5$ as clearance approaches zero:

$$P_{\text{prox}} = \begin{cases} 0, & c \geq d_{\text{prox}}, \\ P_{\max} \left(1 - \frac{c}{d_{\text{prox}}}\right)^2, & c < d_{\text{prox}}, \end{cases} \quad (4.6)$$

where c is the minimum clearance between the rendered vehicle footprint and the lane-boundary/obstacle occupancy grid. In the no-obstacle lane-following environment this discourages driving near lane boundaries; obstacle environments inherit the same term for both lane boundaries and inserted obstacle cells.

Forward lane-following rewards. For forward driving, the terminal reward is

$$R_{\text{term}}^{\text{fwd}} = \begin{cases} 100, & \text{success}, \\ -100, & \text{failure}, \end{cases} \quad (4.7)$$

and the common shaping terms are

$$r_p^{\text{fwd}} = 0.5 \dot{x}, \quad (4.8)$$

$$P_{\text{tractor}} = e_y^2 + e_\psi^2, \quad (4.9)$$

$$P_{\text{trailer}} = (0.5 e_{y,t})^2 + (0.5 e_{\psi,t})^2. \quad (4.10)$$

The four forward reward variants are then

$$R_{\text{dense}}^{\text{fwd}} = r_p^{\text{fwd}} - P_{\text{tractor}} - P_{\text{trailer}} - P_{\text{prox}}, \quad (4.11)$$

$$R_{\text{tractor}}^{\text{fwd}} = r_p^{\text{fwd}} - P_{\text{tractor}} - P_{\text{prox}}, \quad (4.12)$$

$$R_{\text{mult}}^{\text{fwd}} = 4 \exp(-|e_y| - 0.5|e_\psi| - 0.75|e_{y,t}| - 0.5|e_{\psi,t}|) \exp(-1.5|\gamma|) \\ + 0.5 \text{clip}(\dot{x}, 0, 1) - P_{\text{prox}}, \quad (4.13)$$

$$R_{\text{guide}}^{\text{fwd}}(k) = \alpha_k (r_p^{\text{fwd}} - 5\Delta_\delta^2) + (1 - \alpha_k) (r_p^{\text{fwd}} - P_{\text{tractor}} - P_{\text{trailer}}) - P_{\text{prox}}, \quad (4.14)$$

with curriculum weight

$$\alpha_k = \max\left(0, 1 - \frac{k}{T_{\text{guide}}}\right), \quad (4.15)$$

where k is the current training step in the guided transition schedule and T_{guide} is the total number of transition timesteps.

Reverse lane-following rewards. For reverse driving, the terminal reward is

$$R_{\text{term}}^{\text{rev}} = \begin{cases} 200, & \text{success}, \\ -500, & \text{failure}, \end{cases} \quad (4.16)$$

with shared shaping terms

$$r_p^{\text{rev}} = 3 \text{ clip}(-\dot{x}, 0, 1), \quad (4.17)$$

$$P_{\text{path}} = e_y^2 + 0.5 e_\psi^2 + 0.5 e_{y,t}^2 + 0.25 e_{\psi,t}^2, \quad (4.18)$$

$$P_{\text{jack}} = 10 \max(0, |\gamma| - 0.4)^2. \quad (4.19)$$

The reverse variants are

$$R_{\text{dense}}^{\text{rev}} = r_p^{\text{rev}} - P_{\text{path}} - P_{\text{jack}} - P_{\text{prox}}, \quad (4.20)$$

$$R_{\text{no-hitch}}^{\text{rev}} = r_p^{\text{rev}} - P_{\text{path}} - P_{\text{prox}}, \quad (4.21)$$

$$\begin{aligned} R_{\text{mult}}^{\text{rev}} &= 5 \text{ clip}(-\dot{x}, 0, 1) \exp(-|e_y| - 0.5|e_\psi| - 0.75|e_{y,t}| - 0.5|e_{\psi,t}|) \\ &\quad \times \exp(-2|\gamma|) - P_{\text{prox}}, \end{aligned} \quad (4.22)$$

$$R_{\text{guide}}^{\text{rev}}(k) = \alpha_k (r_p^{\text{rev}} - 5\Delta_\delta^2 - P_{\text{jack}}) + (1 - \alpha_k) (r_p^{\text{rev}} - P_{\text{path}} - P_{\text{jack}}) - P_{\text{prox}}. \quad (4.23)$$

Obstacle-aware extensions. Obstacle environments inherit the underlying forward or reverse reward, including the proximity penalty above, and add a difficulty-dependent speed bonus during non-terminal episodes:

$$R_{\text{obs,step}} = R_{\text{base}} + d \left(1 - \frac{|\dot{x}|}{\dot{x}_{\text{max}}} \right) c_{\text{slow}}, \quad (4.24)$$

where $d \in [0, 1]$ is obstacle difficulty and $c_{\text{slow}} = 0.1$. If an obstacle episode terminates through failure before success, the reward is replaced by

$$s(d) = \frac{e^{k_d d} - 1}{e^{k_d} - 1}, \quad k_d = 3, \quad (4.25)$$

$$R_{\text{fail}}^{\text{obs}} = p R_{\text{max}} s(d), \quad (4.26)$$

where $p \in [0, 1]$ is progress along the route and $R_{\text{max}} = 50$ for forward obstacle tasks and $R_{\text{max}} = 100$ for reverse obstacle tasks. If the vehicle instead performs a deliberate stop in front of an obstacle, the terminal reward becomes

$$R_{\text{stop}}^{\text{obs}} = 1.5 p R_{\text{max}} s(d). \quad (4.27)$$

4.8.2 Multiplicative vs. Additive Reward Structures

Table 5.5 and Table 5.6 compare these reward variants directly. The dense and tractor/no-hitch forms are additive penalties, while the multiplicative form forces the agent to reduce tracking and articulation errors jointly instead of collecting partial credit from only one term. The guided reward is a curriculum-shaped interpolation: it initially rewards agreement with a pure-pursuit controller and then decays smoothly into the dense objective.

4.9 Hyperparameter Optimisation

The TD3 agent requires careful hyperparameter tuning. The actor-critic learning rates, replay buffer size, target network update rate, and noise scale are selected over simulation rollouts. For fixed-speed path-tracking experiments the action space is one-dimensional and contains only steering-rate control; when speed remains controllable, exploration noise is kept larger on the speed dimension than on steering. The training pipeline supports matched agents across state-only, lidar, and BEV observation variants. BEV policies use custom multi-input feature extractors, while the state-only and lidar policies use standard multilayer perceptron policies.

To reduce wall-clock cost for image-based training, the training script supports vectorised environment execution with subprocess workers. TD3 optimisation is performed with step-based updates so that the replay buffer and gradient schedule remain responsive for both single-environment and vectorised runs. Vectorised execution is still important for the CNN-based BEV policy because it improves throughput by overlapping environment interaction across subprocess workers, but it no longer changes the underlying update schedule.

4.9.1 Representation Pretraining

Two of the BEV variants use an explicit pretraining stage before TD3 optimisation. The autoencoder-based model transfers a compact pretrained CNN encoder, while the UNet-based model transfers a pretrained UNet encoder after reconstruction training. Both pretraining procedures operate on BEV images collected from policy rollouts and simple geometric controllers. This separates representation learning from control learning and allows direct comparison against the end-to-end CNN baseline. A second set of BEV training diagnostics uses feature scaling: the image branch can be introduced gradually after a state-based policy has begun to solve the task, and the non-steering state inputs can later be reduced to test whether the image representation can replace the engineered tracking state.

4.9.2 Classical Controller Tuning

A reproducible tuning pipeline is used for the classical baselines in later comparison chapters. Forward and reverse pure-pursuit, PID, and MPC controllers are optimised using differential evolution over repeated simulation episodes. The scalar objective penalises trailer cross-track error, jackknife events, collisions, and non-completion. After optimisation, each controller is validated on held-out seeds and a two-dimensional sensitivity sweep is generated over the most influential parameters. This ensures that comparisons against TD3 are made against tuned baselines rather than hand-picked nominal gains.

4.10 Failure Replay Curriculum

A persistent challenge in training RL agents on the reverse lane-following task is that the episode failure rate is high early in training and the lane geometry is re-randomised at every reset. In the standard random-reset regime, each failure is discarded and replaced by a fresh, randomly drawn layout. As a result, the agent rarely encounters the same hard configuration twice, reducing the pressure to master any particular geometry and potentially slowing convergence on the inherently open-loop-unstable reverse task.

To address this, a *failure replay curriculum* is implemented as a transparent environment wrapper. At each episode boundary the wrapper inspects the episode outcome: if the episode terminated due to a failure—jackknife, collision, or out-of-bounds departure before reaching the goal—the next reset reuses the same random seed that generated the current path and spawn position, exactly replaying the same scenario. If the episode was completed successfully or timed out, a fresh seed is drawn and a new random layout is used. A configurable consecutive-retry ceiling (default five retries) prevents the agent from being trapped indefinitely on a pathologically difficult layout.

This strategy is a form of automatic curriculum learning: rather than constructing a manually graded task sequence, the curriculum emerges from the agent’s own failure signal. It is implemented orthogonally to the reward formulation and observation representation ablations. The wrapper modifies only the episode-level sampling policy—not the per-step reward signal or the information available to the agent—and is therefore controlled by a separate training flag. Models trained with and without failure replay are saved to distinct directories to ensure clean comparison.

The curriculum is most meaningful for scenarios where the per-episode failure rate is high and layout difficulty is non-uniform, namely reverse lane-following and reverse obstacle avoidance. For forward lane-following the failure rate is low after minimal training and the benefit is expected to be negligible.

Chapter 5

Results and Discussion

5.1 Path Following Performance

The results pipeline supports controlled comparison across observation modalities and controller families. Forward lane-following serves as the first benchmark task, with matched TD3 agents trained on state-only, lidar, end-to-end CNN BEV, pretrained autoencoder BEV, pretrained UNet BEV, and diagnostic scaled-CNN BEV observations. The evaluation script aggregates tractor and trailer cross-track error, hitch-angle statistics, completion rate, and inference latency across repeated episodes.

This structure is important for the thesis because it separates two questions that were previously entangled: whether the articulated control task benefits from richer sensing, and whether any benefit comes from the sensing modality itself or from the representation-learning pipeline used to process it.

The experiments are organised in the following order:

1. select the best observation space on forward and reverse lane-following without obstacles;
2. hold that observation fixed and compare reward formulations on the same no-obstacle tasks;
3. benchmark the best TD3 configuration against tuned classical controllers on matched no-obstacle scenarios;
4. evaluate the failure replay curriculum (Section 4.10) on the reverse task, holding observation and reward fixed, to isolate the effect of the reset strategy on convergence and final performance.

This sequence keeps each ablation focused on one design variable at a time and ensures that the final controller benchmark is performed on the cleanest apples-to-apples task definition. In particular, the no-obstacle benchmark is run at fixed speed for every

method, so that differences in performance are attributed to path tracking and articulation stabilisation rather than to longitudinal speed selection.

Table 5.1 converts that sequence into an executable run matrix. The training launcher supports all observation and reward combinations listed there through the `scenario`, `obs`, and `reward` arguments, while the benchmark scripts are used only after model selection.

Table 5.1: Planned run matrix for the experimental campaign

Study	Scenario	Observation sweep	Reward sweep	Runs	Purpose
Phase 1A	forward	state, lidar, BEV variants	dense	5+	Select best forward observation space and BEV representation
Phase 1B	reverse	state, lidar, BEV variants	dense	5+	Select best reverse observation space and BEV representation
Phase 2A	forward	best from Phase 1A	dense, tractor_focus, multiplicative	3	Select best forward reward
Phase 2B	reverse	best from Phase 1B	dense, no_hitch, multiplicative	3	Select best reverse reward
Phase 3	forward, reverse	best RL configuration per direction	benchmark only	2	Compare TD3 against tuned pure pursuit, PID, and MPC
Phase 4	forward_obs, reverse_obs	lidar or BEV	best reward from matching direction	4	Secondary obstacle-navigation study

5.2 Observation Space Ablation

The observation space ablation compares the following configurations on the no-obstacle forward and reverse tasks: state-only, lidar (best beam count from Section 5.3), end-to-end CNN BEV from random initialisation, diagnostic BEV state-only through the multi-input policy, scaled-CNN BEV, frozen autoencoder BEV, fine-tuned autoencoder BEV, frozen UNet BEV, and fine-tuned UNet BEV. All agents share the same TD3 backbone, reward formulation, and hyperparameters wherever possible, so performance differences are attributed to the observation and representation choices alone. The diagnostic BEV state-only and scaled-CNN variants are included because early experiments showed that a randomly initialised image branch can degrade an otherwise learnable state policy. These runs distinguish a failure of the visual representation from a failure of the BEV environment or Stable Baselines3 multi-input wrapper.

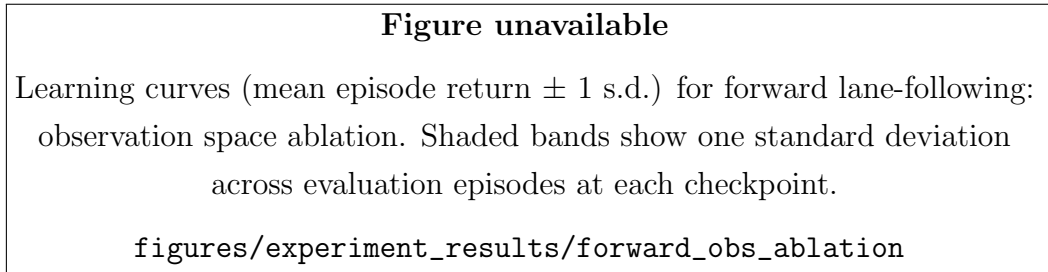


Figure 5.1: Learning curves (mean episode return ± 1 s.d.) for forward lane-following: observation space ablation. Shaded bands show one standard deviation across evaluation episodes at each checkpoint.

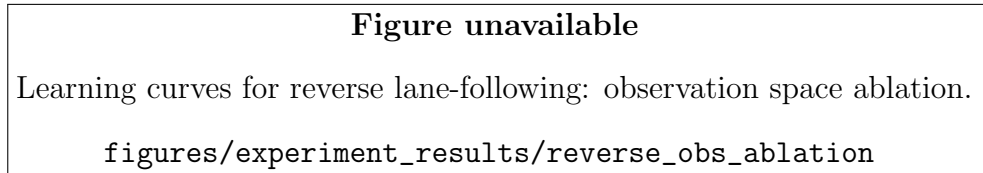


Figure 5.2: Learning curves for reverse lane-following: observation space ablation.

Table 5.2: Forward lane-following: observation space ablation (dense reward, no obstacles, 30 eval episodes).

Observation	CTE Tractor (m)	CTE Trailer (m)	Max $ \gamma $ (°)	Completion (%)	Jackknife (%)	Inference (ms)
State (MLP)	0.921	0.428	14.5	0.0	0.0	<1
Lidar-16 (MLP)	1.083	0.610	9.9	0.0	0.0	<1
BEV CNN (scratch)	0.939	0.496	12.7	0.0	0.0	<1
BEV AE (frozen)	0.834	0.892	35.0	0.0	0.0	<1
BEV AE (unfrozen)	0.719	0.709	30.4	0.0	0.0	<1
BEV UNet (frozen)	0.891	0.523	15.5	0.0	0.0	1.47
BEV UNet (unfrozen)	—	—	—	—	—	—

Table 5.3: Reverse lane-following: observation space ablation (dense reward, no obstacles, 30 eval episodes).

Observation	CTE Tractor (m)	CTE Trailer (m)	Max $ \gamma $ (°)	Completion (%)	Jackknife (%)	Inference (ms)
State (MLP)	0.911	1.516	44.3	0.0	3.3	<1
Lidar-16 (MLP)	0.942	1.065	16.3	0.0	0.0	<1
BEV CNN (scratch)	0.535	1.738	7.7	0.0	0.0	<1
BEV AE (frozen)	—	—	—	—	—	—
BEV AE (unfrozen)	—	—	—	—	—	—
BEV UNet (frozen)	—	—	—	—	—	—
BEV UNet (unfrozen)	—	—	—	—	—	—

5.3 Lidar Beam-Count Ablation

Lidar is treated separately from image-based methods because it is a qualitatively different sensing modality: a sparse one-dimensional range scan rather than a two-dimensional bird’s-eye-view image. The beam-count ablation isolates the effect of lidar resolution on path-tracking performance and inference cost, which trade off directly: more beams supply finer obstacle geometry but enlarge the observation vector and the policy network. Four beam counts are compared, $\{8, 16, 24, 32\}$, on the forward no-obstacle task with the dense reward formulation. The 24-beam configuration is of particular interest because it matches the lidar resolution carried into the hardware deployment (Chapter 6), so the

ablation doubles as a check that the deployed resolution sits on the favourable part of this trade-off rather than being chosen arbitrarily.

Table 5.4: Lidar beam-count ablation on forward lane-following (dense reward, no obstacles, 30 eval episodes).

Beams	CTE Trailer mean (m)	CTE Trailer RMS (m)	Max $ \gamma $ ($^{\circ}$)	Completion (%)	Jackknife (%)	Inference (ms)
4	—	—	—	—	—	—
8	0.323	0.347	8.2	0.0	0.0	<1
16	0.610	0.745	9.9	0.0	0.0	<1
24	0.353	0.378	8.1	0.0	0.0	<1
32	0.401	0.428	9.0	0.0	0.0	<1

5.4 Reward Formulation Ablation

The reward ablation holds the observation space fixed at the best configuration from Section 5.2 and compares reward formulations. Four variants are evaluated on forward lane-following and four on reverse lane-following.

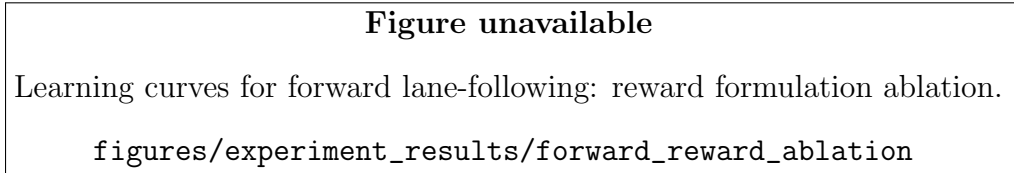


Figure 5.3: Learning curves for forward lane-following: reward formulation ablation.

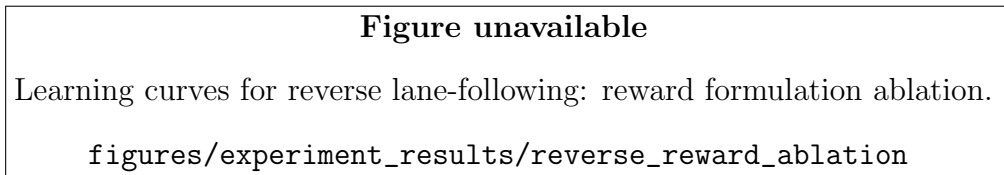


Figure 5.4: Learning curves for reverse lane-following: reward formulation ablation.

Table 5.5: Forward lane-following: reward formulation ablation (lidar-24 obs, no obstacles, 30 eval episodes).

Reward	CTE Tractor (m)	CTE Trailer (m)	Max $ \gamma $ ($^{\circ}$)	Completion (%)	Jackknife (%)	Total reward
Dense	0.281	0.353	8.1	0.0	0.0	621.07
Tractor focus	0.283	0.362	8.1	0.0	0.0	629.40
Multiplicative	0.297	0.378	8.5	0.0	0.0	656.94
Guided	0.640	0.609	14.8	0.0	0.0	621.10

Table 5.6: Reverse lane-following: reward formulation ablation (lidar-24 obs, no obstacles, 30 eval episodes).

Reward	CTE Tractor (m)	CTE Trailer (m)	Max $ \gamma $ ($^{\circ}$)	Completion (%)	Jackknife (%)	Total reward
Dense	1.015	1.037	22.4	0.0	0.0	-944.52
No hitch	0.722	0.591	23.3	0.0	0.0	-749.19
Multiplicative	0.845	0.977	18.2	0.0	0.0	164.40
Guided	1.105	1.358	14.2	0.0	0.0	397.16

5.5 Statistical Rigor and Generalisation

To avoid over-interpreting single aggregate scores, the evaluation protocol is designed to report more than a point estimate for each ablation and benchmark comparison: the number of training seeds, the number of held-out evaluation scenarios, and a measure of uncertainty such as standard deviation or confidence intervals. Where two methods are close in mean performance, effect sizes or simple statistical tests are intended to separate meaningful differences from the noise induced by stochastic training and random scenario generation. This matters acutely for articulated control, where training is high-variance and a gap of a few centimetres in mean cross-track error can lie well within run-to-run scatter; reporting only a single best number would risk reading such scatter as a real effect.

Generalisation is treated as something to be measured rather than assumed. In the present simulator, this means constructing held-out benchmark sets that differ from the training distribution in controlled ways, for example through tighter curvature segments, different spawn offsets, unseen reverse initialisation errors, denser obstacle placements, or

degraded observations. A controller that performs well only on nominal paths does not provide strong evidence of robustness for articulated autonomy, where the failure modes of interest—jackknifing, loss of trailer tracking—tend to appear precisely at the edges of the training distribution. The protocol therefore distinguishes *in-distribution* benchmark performance from *out-of-distribution* robustness, so that the two are reported separately rather than conflated.

5.6 Training Strategy: Failure Replay Curriculum

The failure replay curriculum (Section 4.10) is evaluated as a controlled ablation on the reverse lane-following task, where the episode failure rate is highest and the effect of replay is therefore expected to be most pronounced. Two agents are trained with identical hyperparameters, observation spaces, and reward formulations, differing only in whether the reset strategy replays failed scenarios or draws a fresh random layout.

The primary comparison metric is the learning curve: mean episode return plotted against training timesteps, together with the rate at which episodes terminate in failure (jackknife or out-of-bounds). A curriculum that accelerates convergence is expected to produce a steeper early rise in return and a faster decline in the failure rate during the phase when the policy is least stable, reaching a given level of performance in fewer timesteps than the random-reset baseline. The reverse task is the natural place to look for this effect: because the open-loop reverse dynamics are unstable (Chapter 3), early policies fail often, and it is exactly these early failures that the replay curriculum recycles into additional learning signal.

Final performance (mean absolute trailer CTE, jackknife rate, and $\max |\gamma|$ over held-out evaluation seeds) is also compared, as the curriculum may improve sample efficiency without necessarily raising the asymptotic performance ceiling. Results are evaluated on fixed held-out seeds that were not seen during training, so that any advantage from repeated exposure to particular layouts during training does not inflate the reported evaluation metrics.

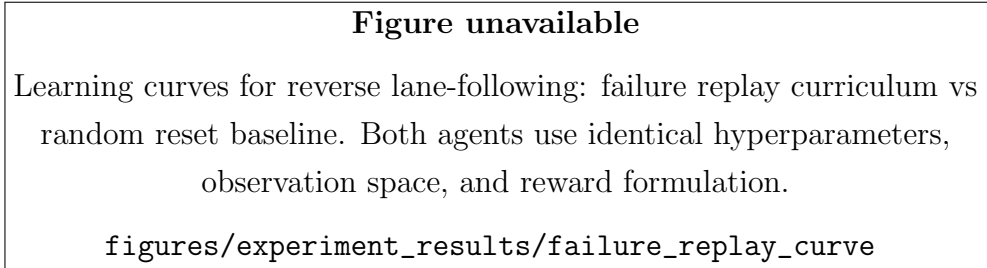


Figure 5.5: Learning curves for reverse lane-following: failure replay curriculum vs random reset baseline. Both agents use identical hyperparameters, observation space, and reward formulation.

Table 5.7: Failure replay curriculum ablation on reverse lane-following (lidar-24 obs, dense reward, 30 held-out eval episodes).

Reset strategy	Steps to 50% completion ($\times 10^3$)	CTE Trailer (m)	Max $ \gamma $ ($^\circ$)	Completion (%)	Jackknife (%)
Random reset	—	1.037	22.4	0.0	0.0
Failure replay	—	—	—	—	—

5.7 Reverse Maneuvering Success

Reverse evaluation uses dedicated reverse line-following and reverse obstacle-avoidance environments, along with reverse pure-pursuit, reverse PID, and reverse MPC baselines. All reverse controllers are formulated around hitch-angle stabilisation and trailer-referenced tracking errors, reflecting the fact that the trailer leads the manoeuvre during backing. This provides a consistent basis for comparing learned and analytical controllers on the inherently unstable reverse articulated-driving problem.

5.8 Perception Robustness

The perception study uses a concrete ablation structure: state variables only, state plus lidar, end-to-end BEV learning, scaled BEV learning, and pretrained BEV encoders. Because all variants share the same TD3 backbone and task definition, differences in tracking performance can be attributed more directly to the observation and representation choices than to changes in the control architecture. The scaled-CNN experiments are interpreted as representation-learning diagnostics: if a policy learns when the image branch is initially suppressed but degrades when the image contribution reaches full scale, the result indicates that the learned visual features are not yet as reliable as the engineered state variables.

In practical terms, the most meaningful observation-space study is the one run on the no-obstacle environments. Obstacle environments confound the comparison because different observation spaces change not only path-tracking quality but also how much implicit local-planning information is available to the policy. They also change how much benefit a method can obtain from modulating speed, which is undesirable in the core controller comparison.

5.9 Obstacle Navigation

Obstacle-avoidance results are reported separately from the no-obstacle benchmark because the RL agent learns a joint local-planning and control policy (including speed modulation), while the classical baselines are pure path-tracking controllers. The comparison is therefore not apples-to-apples and is interpreted as an end-to-end navigation study rather than a controller-only comparison.

Table 5.8: Forward obstacle-avoidance: TD3 performance by observation space (dense reward, 5–10 obstacles per episode, 30 eval episodes).

Observation	CTE Trailer (m)	Max $ \gamma $ (°)	Path completion (%)	Collision (%)	Jackknife (%)	Min clearance (m)	Inference (ms)
State (MLP)	0.447	7.2	0.0	100.0	0.0	—	<1
Lidar-16 (MLP)	0.501	8.7	0.0	100.0	0.0	—	<1
BEV CNN (scratch)	0.537	16.5	0.0	100.0	0.0	—	<1

Table 5.9: Reverse obstacle-avoidance: TD3 performance by observation space (dense reward, 5–10 obstacles per episode, 30 eval episodes).

Observation	CTE Trailer (m)	Max $ \gamma $ (°)	Path completion (%)	Collision (%)	Jackknife (%)	Min clearance (m)	Inference (ms)
State (MLP)	1.022	10.9	0.0	96.7	0.0	—	<1
Lidar-16 (MLP)	0.512	7.0	0.0	96.7	0.0	—	<1
BEV CNN (scratch)	—	—	—	—	—	—	—

5.10 Benchmark Comparison

Table 5.10: Forward lane-following: controller benchmark (no obstacles, 50 eval episodes, held-out seeds).

Controller	CTE Trailer mean (m)	CTE Trailer RMS (m)	Max $ \gamma $ (°)	Completion (%)	Jackknife (%)	Latency (ms)
TD3 (best config)	0.371	0.402	8.8	0.0	0.0	<1
Forward pure pursuit	0.335	0.363	9.5	0.0	0.0	<1
PID (tuned)	0.332	0.361	13.9	0.0	0.0	<1
MPC (tuned)	0.379	0.411	16.4	0.0	0.0	1.17

Table 5.11: Reverse lane-following: controller benchmark (no obstacles, 50 eval episodes, held-out seeds).

Controller	CTE Trailer mean (m)	CTE Trailer RMS (m)	Max $ \gamma $ ($^{\circ}$)	Completion (%)	Jackknife (%)	Latency (ms)
TD3 (best config)	1.023	1.101	22.1	0.0	0.0	<1
Reverse pure pursuit	0.988	1.125	54.5	0.0	0.0	<1
PID reverse (tuned)	0.984	1.123	54.9	0.0	0.0	<1
MPC reverse (tuned)	0.500	0.520	55.4	0.0	0.0	1.45

Benchmarking is performed on pre-generated scenario sets so that TD3 and the classical baselines face identical lane geometry and obstacle placement. Classical controllers are tuned before comparison using differential evolution and then validated on held-out seeds. Each benchmark episode logs both control quality and computational cost, allowing comparison not only of tracking error but also of completion rate, jackknife frequency, collision rate, obstacle clearance, and controller latency.

The results must be understood in terms of multi-objective performance: in rigid-body systems, success is synonymous with minimising cross-track error. In articulated systems, hitch stability takes precedence over exact path following. An agent that maintains $|\gamma| < 5^{\circ}$ throughout a reverse task is more successful than one that achieves lower CTE but triggers jackknife events. For that reason, the benchmark framework reports trailer tracking, articulation stability, and completion together rather than reducing performance to a single scalar.

For obstacle scenarios, the benchmark question changes. An RL obstacle policy is effectively learning a joint local-planning and control policy, including speed modulation, whereas pure-pursuit, PID, and MPC are primarily path-tracking controllers. Obstacle-task results are therefore not presented as directly comparable to controller-only baselines unless a matched planning layer and longitudinal policy are added to those baselines. Obstacle results are reported either as a separate end-to-end navigation study or as a comparison against planner+controller classical stacks.

5.11 Failure Analysis and Deployment Relevance

Average metrics alone are insufficient for articulated-vehicle control, because the operational significance of a failure depends strongly on how that failure occurs. The analysis therefore examines representative failure cases for each controller family—such as gradual corner-cutting, growing lateral oscillations, abrupt jackknife onset, and obstacle-clearance failures—rather than mean error alone. Trajectory overlays and hitch-angle traces are

the natural diagnostics here, because they reveal whether an error mode is primarily geometric, dynamic, or perception-driven, and therefore which part of the pipeline a fix should target.

This failure-oriented discussion also connects the simulation study to deployment relevance. A controller that attains a low mean cross-track error but exhibits rare catastrophic jackknife episodes is less deployable than a controller with slightly higher average error but consistent stability margins. For the same reason, controller latency and architectural modularity are discussed alongside tracking performance. These considerations help position the thesis as a study in autonomous articulated-system design rather than only an optimisation exercise inside a simulator.

Table 5.12: Recommended experiment matrix for the thesis

Experiment	Task	Observation	Success Metric
Observation ablation (forward)	Path following	state, lidar, BEV variants	Fixed-speed trailer mean abs. CTE, max $ \gamma $, completion
Observation ablation (reverse)	Reverse tracking	state, lidar, BEV variants	Fixed-speed trailer mean abs. CTE, max $ \gamma $, jackknife rate
Reward ablation (forward)	Path following	best observation only	Fixed-speed mean return, trailer CTE, stability metrics
Reward ablation (reverse)	Reverse tracking	best observation only	Fixed-speed completion, jackknife rate, trailer CTE
Controller benchmark (forward)	Path following	best RL obs. vs. classical	Fixed-speed TD3 vs. pure pursuit vs. PID vs. MPC
Controller benchmark (reverse)	Reverse tracking	best RL obs. vs. classical	Fixed-speed TD3 vs. reverse pure pursuit vs. reverse PID vs. reverse MPC
Obstacle RL extension	End-to-end navigation	lidar or BEV	Variable-speed collision rate, completion, min clearance
Obstacle classical comparison	Planner + controller	matched local path + controller	Only if classical baselines receive equivalent planning information

Chapter 6

Sim-to-Real Deployment

6.1 Overview

The preceding chapters developed and evaluated an articulated tractor-trailer controller entirely within a 2D Pygame/Gymnasium simulation environment. This chapter describes the transfer of that controller to a physical robotic platform, constituting the sim-to-real component of the thesis. Unlike a conventional model-based stack, the deployed system does not re-implement a hand-derived tracking law on the vehicle; instead, the simulation-trained reinforcement-learning (RL) policies are executed directly on hardware through a thin bridging layer that reproduces, in real time, the observation and action interfaces the policies were trained against. The objective is to demonstrate that the vehicle model, the observation definitions, and the learned control logic established in simulation can be ported to real hardware with minimal algorithmic redesign, and to characterise the engineering challenges encountered during that transfer.

The deployment is built on a deliberately stripped-down Autoware stack. The standard Autoware localisation and perception pipelines are retained essentially unchanged, while the planning and control layers are replaced by a custom RL bridge that runs the forward and reverse policies. Two pieces of supporting infrastructure are described in detail because they are specific to the articulated, sensor-light platform: an onboard LiDAR-based hitch-angle estimator that recovers the articulation angle without any trailer-mounted sensor, and a Smith predictor that compensates for actuator delay so that delay-free-trained policies behave correctly on a lagging physical actuator.

6.2 Hardware Platform

6.2.1 Tractor Vehicle

The physical tractor is an Agilex Hunter SE, a compact electric wheeled unmanned ground vehicle with an Ackermann steering geometry. The relevant mechanical parameters are a

wheelbase of 0.65 m, a wheel radius of 0.15 m, and a maximum steering angle of 0.436 rad (25°). Vehicle actuation is commanded over a CAN bus on a 20 ms cycle; speed commands are encoded as 16-bit signed integers in units of 1 mm/s (the commanded speed in m/s scaled by 1000) and steering commands as 16-bit signed integers scaled by approximately 1320 counts per radian (≈ 0.8 mrad per count).

6.2.2 Trailer

The trailer is coupled to the tractor through a revolute hitch joint located at (or marginally behind) the tractor rear axle, matching the training-time trailer model in which the hitch coincides with the rear axle. The lab trailer has a wheelbase of approximately 2 m and a body width of roughly 0.33 m¹. The articulation angle γ , defined as the difference between the tractor and trailer headings ($\gamma = \psi_{\text{truck}} - \psi_{\text{trailer}}$), is *estimated onboard* from the tractor’s own LiDAR rather than measured by any trailer-mounted sensor; the procedure is detailed in Section 6.3.4. The estimate is published to the ROS2 network on the `/vehicle/trailer_state` topic; the control bridge consumes the most recent published value, with no staleness gating currently applied. Two distinct angular limits appear in the stack and should not be conflated: the onboard estimator clamps the *published* articulation estimate to $|\gamma| \leq 90^\circ$, whereas the deployment trailer model defines a tighter operational limit of $|\gamma| \leq 80^\circ$ (1.40 rad), itself below the 90° jackknife condition used to terminate simulation episodes.

6.2.3 Sensor Suite

The vehicle is instrumented with the following sensors:

- **LiDAR:** RoboSense Helios mechanical LiDAR mounted at the front of the tractor, publishing 3D point clouds on `/rslidar_points`. This single sensor is reused for three purposes: NDT-based localisation, environment perception, and onboard hitch-angle estimation.
- **Cameras:** Six Basler Pylon industrial cameras (rear, rear-right, centre, left, merging, and right) producing compressed colour images. The cameras are not consumed by the LiDAR-based RL policies and are used for logging and monitoring only.
- **IMU:** WitMotion 6-DoF inertial measurement unit, with a gyroscope-bias estimator applied in the localisation pipeline.

¹The trailer wheelbase is recorded in the deployment configuration as a nominal 2 m pending an exact in-person measurement; controller and estimator parameters are not sensitive to small errors in this value.

- **GNSS:** u-blox ZED-F9P receiver with RTK corrections, providing centimetre-level position estimates converted to MGRS coordinates for Autoware localisation.
- **Wheel odometry:** Vehicle speed and steering-angle feedback recovered from CAN messages and fused with IMU data for dead-reckoning between GNSS and scan-matching updates.

6.3 Software Architecture

6.3.1 Middleware and Framework

The deployment stack runs on ROS2 Humble (Ubuntu 22.04) and is integrated into the Autoware autonomous-driving framework. Autoware provides modular localisation, perception, planning, and control pipelines whose interfaces are defined by standardised ROS2 message types. The deployment in this thesis retains the standard Autoware sensing, localisation, and perception modules but substitutes the planning and control layers with a learned RL bridge, so that the vehicle is driven by the same policies evaluated in simulation rather than by an analytical controller.

6.3.2 Relationship to the Conventional Articulated Pipeline

For context, Autoware’s conventional approach to articulated vehicles is fully model-based: a truck-trailer mode manager routes goals to a freespace planner (a hybrid A* search that respects the non-holonomic articulation constraint at the hitch), and the resulting reference trajectory is tracked by a model-predictive or LQR lateral controller. This model-based pipeline exists within the wider project and represents the standard, well-understood baseline for trailer control; the relevant control background is reviewed in Chapter 2. The contribution of *this* thesis, however, is the learned controller, and the deployment described here therefore routes control through the RL bridge instead of the planner/MPC chain. The two are mutually exclusive at run time and are selected at launch.

6.3.3 RL Bridge Node Architecture

The learned control path comprises the following principal nodes:

`lane_reference_node` Loads the Lanelet2 map, selects the active lane from the ego odometry and the current goal, and publishes the lane centreline as a reference path together with a `drive_enabled` flag and a `drive_direction` flag (forward or reverse). This node supplies the path geometry against which the policy’s tracking observation is computed.

rl_bridge_node The core of the deployment. It loads the trained forward and reverse TD3 policies (Stable-Baselines3 / PyTorch checkpoints) and runs a 10 Hz control loop. On each tick it assembles the policy observation in exactly the layout used during training — an 8-dimensional state vector (steering angle, articulation angle, tractor and trailer tracking errors, and look-ahead path curvature) concatenated with a 24-beam LiDAR range vector — queries the active policy deterministically, and converts the 2-dimensional action (steering rate and commanded speed) into the steering-angle and acceleration commands expected by the vehicle interface. Direction-dependent post-processing (action smoothing and kinematic rate scaling) is applied for the reverse policy, whose outputs are inherently more oscillatory than the forward policy's.

hitch_angle_estimator Estimates the articulation angle from the tractor LiDAR and publishes it on `/vehicle/trailer_state` (Section 6.3.4).

initialpose_shim Bridges the RViz `/initialpose` message to the Autoware pose-initialiser service, allowing the operator to seed localisation interactively.

controller_canbridge Translates `autoware_control_msgs/Control` commands into Hunter SE CAN frames and parses vehicle feedback (velocity, steering angle) back into ROS2 topics.

The forward and reverse policies are loaded from separate checkpoints and selected according to the `drive_direction` flag, so that the same bridge handles both manoeuvring regimes without retraining or reconfiguration.

6.3.4 Hitch Angle Estimation

Because the trailer carries no articulation encoder and no trailer-mounted sensing, the articulation angle is recovered geometrically from the tractor's front LiDAR, which observes the front (and, at larger angles, the side) face of the trailer. The estimator operates on each incoming point cloud as follows.

Region of interest. Points are first cropped to a box in the LiDAR frame that brackets the expected location of the trailer face. The lateral extent of this region is shifted to track the face as the trailer swings: using the current filtered estimate γ , the lateral offset is set to

$$\Delta y = -D_{\text{face}} \sin \gamma, \quad (6.1)$$

where D_{face} is the approximate longitudinal distance from the hitch pivot to the face. This keeps the cropped points on the trailer face across the full range of articulation rather than only near $\gamma = 0$.

Rectangle fitting. Rather than fitting a single planar normal, the estimator fits the orientation of the trailer body to the cropped points using the minimum-closeness rectangle criterion of Zhang *et al.* [1]. For a candidate orientation θ , the points are rotated into axis-aligned coordinates (u, v) and scored by their closeness to the nearest edges of the bounding rectangle,

$$C(\theta) = \sum_i \min(d_i^u, d_i^v)^2, \quad d_i^u = \min(|u_i - u_{\min}|, |u_i - u_{\max}|), \quad (6.2)$$

with d_i^v defined analogously. The fitted orientation minimises $C(\theta)$ augmented by a dimension-prior penalty that favours rectangles consistent with the known trailer width and length:

$$\theta^* = \arg \min_{\theta \in [0, \pi)} [C(\theta) + \lambda P_{\text{dim}}(\theta)]. \quad (6.3)$$

This formulation is robust to whether only the front face, both the front and a side face (an “L” shape), or only a side face is visible — the dominant cause of failure for a naive single-face normal estimate at large articulation. The four 90° heading candidates implied by θ^* are disambiguated by choosing the one closest to the previous filtered estimate and most consistent with the dimension prior, yielding a trailer longitudinal direction $\hat{\mathbf{d}}_t = (\cos \theta^*, \sin \theta^*)$.

Articulation angle and filtering. The raw articulation angle is the signed angle between the truck-forward axis $\hat{\mathbf{x}} = (1, 0)$ and $\hat{\mathbf{d}}_t$,

$$\tilde{\gamma} = \text{atan2}(\hat{x}_1 \hat{d}_{t,2} - \hat{x}_2 \hat{d}_{t,1}, \hat{\mathbf{x}} \cdot \hat{\mathbf{d}}_t), \quad (6.4)$$

to which a field-calibration sign s and offset γ_0 are applied and the result clamped to the operational range, $\gamma = \text{clip}(s \tilde{\gamma} + \gamma_0, -\gamma_{\max}, \gamma_{\max})$. The sign convention is chosen so that the published value matches Autoware’s $\gamma = \psi_{\text{truck}} - \psi_{\text{trailer}}$. The raw per-scan estimate is then passed through a constant-angular-velocity Kalman filter on the state $[\gamma, \dot{\gamma}]$, with Mahalanobis innovation gating to reject outlier scans, and a short moving-average is maintained in parallel as a diagnostic. The filtered angle and its rate populate the `hitch_angle` and `hitch_rate` fields of the published `TrailerState`.

This onboard, geometry-plus-filter estimator is the actual mechanism behind the `/vehicle/trailer_state` signal; it is not an external measurement. Its noise characteristics, and the way they are handled, are revisited in the gap analysis (Section 6.4.2).

6.3.5 Trailer State Estimation

Because the trailer carries no independent localisation, its full pose is reconstructed from the tractor’s localised kinematic state and the estimated articulation angle. Given the

tractor’s rear-axle position and heading ($\mathbf{p}_{\text{truck}}, \psi_{\text{truck}}$) from `/localization/kinematic_state`, the hitch-point position is

$$\mathbf{p}_{\text{hitch}} = \mathbf{p}_{\text{truck}} + d_h \begin{bmatrix} \cos \psi_{\text{truck}} \\ \sin \psi_{\text{truck}} \end{bmatrix}, \quad (6.5)$$

where d_h is the (small, near-zero) signed hitch offset relative to the tractor rear axle. The trailer heading follows directly from the articulation angle, $\psi_{\text{trailer}} = \psi_{\text{truck}} - \gamma$, and the trailer rear axle is located at

$$\mathbf{p}_{\text{trailer}} = \mathbf{p}_{\text{hitch}} - L_t \begin{bmatrix} \cos \psi_{\text{trailer}} \\ \sin \psi_{\text{trailer}} \end{bmatrix}, \quad (6.6)$$

where $L_t \approx 2 \text{ m}$ is the trailer wheelbase. This reconstruction mirrors the hitch-kinematics model derived in Chapter 3, confirming that the simulation formulation is directly reusable in the deployment stack, and it provides the trailer-referenced tracking errors that form part of the policy observation.

6.3.6 Actuator-Delay Compensation: Smith Predictor

The simulation trains the policies against an idealised actuator that responds instantaneously to commands, whereas the physical steering and drive systems exhibit a first-order lag of the order of 50–150 ms. A policy trained for instantaneous response will, when confronted with a delayed actuator, repeatedly over-correct and excite a lag-induced oscillation. To reconcile the two, a Smith-predictor-style compensator is implemented that lets the delay-free-trained policy act on a delay-compensated state estimate.

The compensator maintains two “shadow” instances of the same kinematic `StateSpaceTractorTrailer` model used in simulation: a *delay-free* shadow with zero actuation time constant, and a *delayed* shadow whose time constant matches the measured hardware lag. On each control tick both shadows are advanced with the command currently propagating through the actuator pipeline. The discrepancy between the measured vehicle state and the delayed shadow forms a residual,

$$\mathbf{r}_k = \mathbf{x}_k^{\text{meas}} - \mathbf{x}_k^{\text{delayed}}, \quad (6.7)$$

which captures model mismatch beyond pure delay (integration scheme, ground-contact effects, friction). The state presented to the policy is the delay-free shadow corrected by this residual,

$$\hat{\mathbf{x}}_k = \mathbf{x}_k^{\text{free}} + \mathbf{r}_k, \quad (6.8)$$

so that the policy “sees” an effectively delay-free vehicle while the residual keeps the prediction anchored to reality. The shadows are re-synchronised to the measured state on reset and whenever a large state jump is detected. The compensator is implemented

and available but is disabled by default, as the shadow time constants require per-platform tuning before the loop can be closed in anger; combined with the explicit steering-rate limit it forms the primary mitigation for the actuation gap discussed in Section 6.4.1.

Learned dynamics model (in progress). The Smith predictor’s fidelity is bounded by the accuracy of the analytical kinematic shadow. Work is in progress to replace, or augment, that shadow with a neural-network approximation of the true system dynamics. A data pipeline converts recorded driving logs (rosbags) into time-aligned 10 Hz state-action-next-state datasets — deriving the trailer pose from the truck pose and the estimated hitch angle where it is not directly recorded — and a multilayer-perceptron dynamics model (**DynamicsMLP**) is trained on these datasets. The learned model is evaluated both by one-step prediction error against held-out logs and by closed-loop rollout divergence from the kinematic model, the latter serving to quantify and localise the sim-to-real dynamics gap. This component is not yet part of the live control loop and is reported here as ongoing work rather than a validated result.

6.4 Sim-to-Real Gap Analysis

Transferring a simulation-trained policy to physical hardware introduces discrepancies between the simulated and real-world dynamics. The following gap sources were identified and addressed during the deployment process.

6.4.1 Actuation Dynamics

The simulation model assumes instantaneous steering response, whereas the physical vehicle exhibits a first-order steering lag introduced by the CAN command latency and the Hunter SE’s internal steering servo. This is the highest-impact discrepancy for a policy trained on instantaneous actuation. It is mitigated by the Smith-predictor compensator of Section 6.3.6, which presents a delay-compensated state to the policy, and by a steering-rate limit applied to the commanded output, consistent with the $\dot{\delta}_{\max}$ constraint used during simulation training.

6.4.2 Hitch-Angle Sensing

In simulation, the articulation angle γ is available exactly from the vehicle state. On hardware it is estimated from LiDAR returns and is therefore subject to noise from sparse or partial returns on the trailer face, mis-segmentation of the region of interest, and momentary loss of the face at large articulation. These error modes are addressed within the estimator itself rather than by a downstream filter: the rectangle-fitting formulation (Equation 6.3) is robust to partial face visibility, and the constant-angular-velocity

Kalman filter with innovation gating rejects outlier scans and smooths the published signal. If the trailer face is lost entirely the estimator withholds new estimates, and the bridge continues from the last published value until the face is reacquired; no automatic disengagement on stale data is currently implemented.

6.4.3 Path Distribution

The policies are trained on smooth, spline-generated reference paths, whereas the deployment reference is the centreline of a Lanelet2 lane, which is piecewise-linear and may contain comparatively sharp vertices. Re-sampling the centreline does not fully reconcile the difference in path-curvature distribution between training and deployment, and this mismatch is a contributing factor to any residual tracking error observed on hardware. It is noted here as a characterisation of the gap rather than something fully eliminated in the current deployment.

6.4.4 Jackknife Recovery

The simulation introduced a terminal episode condition when $|\gamma| > 90^\circ$ but provided no explicit recovery mechanism, since the episode simply resets. Rather than adding a separate runtime supervisor on the vehicle, jackknife resilience is instilled during *training*, through two complementary measures. First, a recovery-scenario reset distribution spawns a fraction of reverse episodes already in the near-jackknife region (articulation magnitudes of roughly 0.35–0.80 rad, i.e. 20–45°), so the policy is repeatedly exposed to high-articulation states its nominal training distribution rarely visits and must learn to steer the trailer back toward alignment. Second, an anti-jackknife override is applied *within the training environment*: whenever the articulation exceeds approximately 0.7 rad (40°), the policy’s steering command is replaced by a maximum counter-steer that drives the articulation back toward zero. This prevents the policy from learning that oscillatory, near-jackknife behaviour is survivable, since every dangerous excursion is corrected during training and chattering no longer pays off. No dedicated runtime recovery override is currently part of the deployed bridge; on hardware, jackknife avoidance therefore relies on the behaviour learned through these measures together with the operational $|\gamma| \leq 80^\circ$ limit.

6.4.5 Localisation Dependency

The simulation provides ground-truth vehicle pose at every step. The hardware stack instead depends on the Autoware localisation pipeline, which fuses RTK-GNSS, gyro-based odometry, and LiDAR NDT scan matching through an extended Kalman filter. GNSS outages or localisation divergence therefore represent a failure mode that does not

exist in simulation; the current deployment treats localisation failure as an emergency-stop condition.

6.5 High-Fidelity Geographic Simulation

The 2D Pygame environment of chapter 3 is deliberately abstract: it captures the tractor-trailer kinematics and lane geometry but not the three-dimensional and photometric structure that the deployed perception and localisation stack actually consumes. A complementary, higher-fidelity simulation environment is therefore being developed to exercise the full Autoware stack — LiDAR-based NDT localisation, perception, and the RL bridge — inside a geographically faithful “digital twin” of a real operating site before committing to physical trials.

This environment is built on the Gazebo simulator using an open world-generation toolchain (Rudis Labs). Real locations are reconstructed as textured 3D meshes authored in Blender, whose physically-based materials are baked into texture maps for efficient real-time rendering. Each world is anchored to GIS coordinates through the SDF `spherical_coordinates` element, which fixes the simulation origin to a real latitude, longitude, and elevation; a templated world generator additionally computes the sun’s azimuth and elevation from the site coordinates and a chosen UTC date and time, so that lighting and shadows match the real location and time of day. The `electrans` tractor and trailer models are spawned into these worlds, so that the same articulated vehicle can be driven through a virtual replica of the target environment.

Because the world is georeferenced and rendered in three dimensions, this environment directly targets several of the sim-to-real gaps catalogued above. A LiDAR sensor model produces point clouds against real building and ground geometry, so NDT scan matching and the onboard hitch-angle estimator (section 6.3.4) can be exercised against realistic returns; the geographic anchoring lets the GNSS and map frames be reproduced consistently with the physical site; and the photorealistic rendering makes it possible to evaluate camera-based perception that the abstract 2D environment cannot represent. It is also the natural setting in which to prototype the infrastructure sensor nodes of section 6.6, since virtual LiDAR units can be placed at arbitrary surveyed locations in the georeferenced world. This effort is in progress and is reported here as a planned bridge between the abstract training environment and on-vehicle deployment rather than as a validated result.

6.6 Infrastructure Sensor Nodes

A structural limitation of the onboard sensing arrangement is that the trailer occludes the region directly behind it. The tractor-mounted LiDAR cannot see past the trailer, and

instrumenting the trailer itself is undesirable: it would add cabling, power, calibration, and a wireless link to an otherwise passive, interchangeable body, and would have to be repeated for every trailer. This is most acute during reverse manoeuvring, precisely when visibility of the space behind the trailer matters most.

A planned line of work, not yet implemented, addresses this with *off-vehicle infrastructure sensor nodes*: fixed LiDAR units placed at strategic vantage points throughout the operating area, each providing a bird’s-eye view of obstacles — crucially including the region behind the trailer — without any sensor being mounted on the trailer. Each node would be calibrated into the shared map frame so that its detections can be expressed in the same coordinates as the onboard perception output and fused into Autoware’s perception and occupancy-grid representation. The intended outcome is a persistent, occlusion-resilient view of dynamic and static obstacles around the articulated vehicle that complements the limited onboard field of view.

This section is a placeholder for that work. The detailed node topology, the calibration and time-synchronisation procedure, the fusion strategy with onboard perception, and the experimental evaluation remain to be specified and carried out.

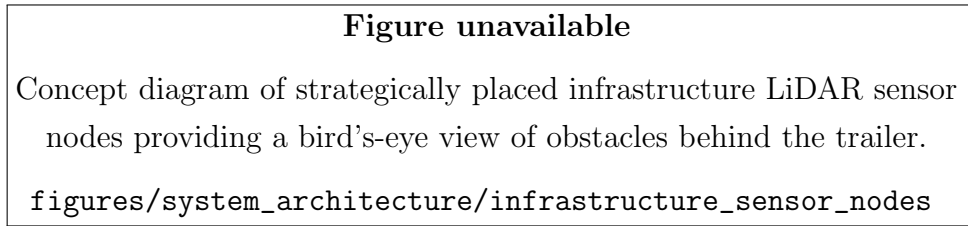


Figure 6.1: Planned infrastructure sensor-node concept: fixed off-vehicle LiDAR units give a bird’s-eye view of obstacles around the articulated vehicle, covering the region behind the trailer that the onboard sensors cannot observe.

6.7 Deployment Results

Hardware validation trials were conducted on [TODO: describe test location and scenario]. The forward policy achieved [TODO: describe forward lane-following behaviour], with a mean trailer cross-track error of [TODO]m and articulation maintained within $|\gamma| < [TODO]^\circ$ throughout nominal operation. Reverse manoeuvring [TODO: describe outcome]. [TODO: report the effect of enabling the Smith predictor on oscillation and tracking once tuned.]

6.8 Summary

This chapter demonstrated that the tractor-trailer control architecture developed in simulation can be ported to a physical platform by executing the trained RL policies directly

through a bridging layer, rather than by re-deriving an analytical controller on the vehicle. The deployment retains Autoware’s standard localisation and perception while replacing planning and control with an RL bridge that runs the forward and reverse policies at 10 Hz over the same observation and action interfaces used in training. The principal engineering contributions are this RL bridge and its integration into a stripped-down Autoware stack, the onboard LiDAR hitch-angle estimator that recovers the articulation angle by rectangle fitting and Kalman filtering without any trailer-mounted sensor, and the Smith-predictor compensator (with an in-progress learned dynamics model) that reconciles delay-free-trained policies with a lagging physical actuator. The gap analysis confirms that the simulation captures the dominant dynamics relevant to controller design, while identifying actuation delay, hitch-angle sensing, path-distribution mismatch, and localisation reliability as the primary points of divergence between the simulated and physical systems. Finally, the planned infrastructure sensor nodes are introduced as a route to occlusion-resilient perception behind the trailer without adding sensors to the trailer itself.

Chapter 7

Conclusion

7.1 Summary of Findings

This thesis presented a reinforcement learning framework for articulated autonomous vehicle control. The key contributions demonstrated are:

- a combined dynamic tractor and kinematic trailer model implemented in a custom Gymnasium simulation environment;
- a vision-based observation space incorporating compact BEV crops derived from the Pygame simulation, enabling spatial perception of lane boundaries and obstacles;
- a TD3 reinforcement learning agent trained on lane-following and obstacle avoidance tasks with reward formulations that penalise cross-track error, heading error, hitch articulation angle, terminal failures, and close proximity to lane boundaries or obstacles;
- comparison of TD3 performance against tuned pure-pursuit, PID, and MPC classical control baselines on matched articulated path-tracking tasks;
- a sim-to-real deployment (Chapter 6) that runs the trained forward and reverse policies directly on a physical Agilex Hunter SE through a stripped-down Auto-ware stack, supported by onboard LiDAR-based hitch-angle estimation and a Smith predictor for actuator-delay compensation.

The integration of visual perception via BEV imagery broadens the control problem beyond compact vector-state tracking. By processing rendered BEV images through CNN, autoencoder, or UNet-style feature extractors, the agent gains spatial context about lane geometry and obstacle positions that vector state observations alone cannot provide. The scaled-CNN experiments further show that the visual branch must be introduced carefully: a randomly initialised image encoder can interfere with early TD3 learning,

while gradually increasing the image contribution provides a cleaner diagnostic of whether the learned representation is becoming useful for control.

From a research-impact perspective, the thesis is strongest when interpreted not only as an RL training exercise but as a reproducible benchmark and systems study for articulated autonomy. The combination of matched observation ablations, tuned classical baselines, fixed-scenario benchmarking, and explicit articulation-stability metrics provides a framework for asking which sensing and control choices are actually useful for tractor-trailer path tracking. This framing is more defensible and more reusable than reporting isolated policy demonstrations.

7.2 Limitations

The study is subject to the following constraints:

- The bulk of the experimental evaluation is conducted within the custom 2D Pygame/-Gymnasium simulation environment. A hardware deployment exists (Chapter 6), but quantitative sim-to-real performance has not yet been characterised, and behaviour in higher-fidelity physics simulators remains untested.
- The 2D simulation approximates the vehicle dynamics but does not capture all effects present in a real tractor-trailer system, such as load transfer, tire saturation at high slip angles, or hitch joint compliance.
- The current framework assumes a fixed payload; hitch dynamics change significantly with varying trailer mass.
- Obstacle avoidance is limited to static obstacles; dynamic obstacle handling is not addressed.
- Generalisation beyond the training distribution has not yet been fully quantified; robustness to sensor degradation and stronger path-distribution shift remains to be measured explicitly. Actuator delay is actively mitigated on hardware by the Smith predictor of Chapter 6, but its effect on closed-loop tracking has not yet been measured.
- The hardware deployment of Chapter 6 runs the trained policies on the Agilex Hunter SE through the ROS2/Autoware integration, but a full characterisation of sim-to-real performance across the complete range of training tasks remains as future work.

7.3 Future Work

Several directions remain open for future investigation:

- **Higher-fidelity simulation:** Validating the trained policies in a physics-based 3D simulator with a detailed model of the tractor-trailer system, including realistic tire and hitch dynamics. This is already underway as the georeferenced Gazebo “digital-twin” environment of section 6.5, which reconstructs real sites from Blender-authored, GIS-anchored worlds and exercises the full Autoware perception and localisation stack; completing it and quantifying the policies’ behaviour within it is the natural next step.
- **Hardening the hardware deployment:** The trained TD3 policies are already deployed directly on the Hunter SE through the RL bridge of Chapter 6, with actuator-delay compensation provided by the Smith predictor. The next steps are to collect quantitative sim-to-real results across the full task set, complete the in-progress learned neural-network system-dynamics model that augments the Smith predictor, introduce domain randomisation during training to narrow the remaining sim-to-real gap, and add a supervised safety-fallback layer.
- **Infrastructure sensing:** Building the off-vehicle infrastructure LiDAR sensor nodes proposed in Chapter 6, which provide an occlusion-resilient bird’s-eye view of obstacles behind the trailer without instrumenting the trailer itself, and fusing their detections into the onboard perception stack.
- **Planner-controller baselines for obstacle tasks:** Comparing obstacle-avoidance RL policies against complete classical navigation stacks, so that end-to-end navigation is evaluated on a fair task definition rather than against controller-only baselines.
- **Robustness benchmarks:** Adding out-of-distribution test sets with stronger curvature, perturbed initial poses, observation noise, and parameter mismatch, so that the thesis can quantify generalisation rather than only nominal performance.
- **Failure-mode analysis:** Building a structured taxonomy of controller failures, including oscillation growth, corner-cutting, hitch divergence, and obstacle-clearance collapse, to support safety-oriented comparison rather than relying on averages alone.
- **Control-aligned visual pretraining:** Pretraining BEV encoders to predict control-relevant quantities such as cross-track error, heading error, hitch angle, curvature, or expected reward, rather than relying only on pixel reconstruction.

- **Sequence-model policies for tractor-trailer control:** Applying transformer-based offline control methods to the articulated vehicle setting, with training data generated from the TD3 expert policy and augmented with high-error recovery trajectories.
- **Multi-trailer systems:** Extending the kinematic model and RL policy to double or B-train configurations.
- **Online adaptation:** Updating the policy in real time to compensate for varying payloads and road conditions.
- **Control Barrier Function overlays:** Implementing CBFs as a hard safety layer that overrides the RL agent when a collision or jackknife is imminent.
- **Dynamic obstacles:** Extending the obstacle avoidance environment to include moving obstacles and integrating predictive collision avoidance.

Appendix A

Simulation Software Architecture

Appendix B

Full Hyperparameter Tables

Table B.1: TD3 Hyperparameters

Hyperparameter	Value
Learning rate	
Batch size	
Replay buffer size	
Policy noise	
Target update rate	
Timesteps	200,000

Appendix C

Additional Plots

Appendix D

Reward Function Derivation

Bibliography

- [1] X. Zhang, W. Xu, C. Dong, and J. M. Dolan, “Efficient L-shape fitting for vehicle detection using laser scanners,” in *2017 IEEE Intelligent Vehicles Symposium (IV)*, pp. 54–59, 2017.